

Tetris: Versatile TFHE LUT and Its Application to FHE Instruction Set Architecture

Ruida Wang^{1,2}, Jikang Bai^{1,2}, Xuan Shen^{1,2}, Xianhui Lu^{*1,2},
Zhihao Li³, Binwu Xiang^{†4}, Zhiwei Wang^{1,2}, Hongyu Wang^{‡5},
Lutan Zhao^{1,2}, Kunpeng Wang^{1,2}, and Rui Hou^{1,2}

¹State Key Laboratory of Cyberspace Security Defense, Institute
of Information Engineering, CAS

²School of Cybersecurity, UCAS

³Digital Technologies, Ant Group

⁴East China Normal University

⁵Shanxi University

Abstract

Fully Homomorphic Encryption (FHE) enables computation over encrypted data, but deployment is hindered by the gap between plaintext and ciphertext programming models. FHE compilers aim to automate this translation, with a promising approach being *FHE Instruction Set Architecture (FHE-ISA)* based on homomorphic look-up-tables (LUT). However, existing FHE LUT techniques are limited to 16-bit precision and face critical performance bottlenecks.

We introduce Tetris, a versatile TFHE LUT framework for high-precision FHE instructions. Tetris incorporates three advances: a) A GLWE-based design with refined noise control, enabling up to 32-bit LUTs; b) A batched TFHE circuit bootstrapping algorithm that enhances the LUT performance; and c) Adaptive parameterization and parallel execution strategy that is optimized for high-precision evaluation.

These techniques deliver: a) The first general FHE instruction set with 16-bit bivariate and 32-bit univariate operations; b) Performance improvements of $2\times \sim 863\times$ over modern TFHE LUT approaches, and $2901\times$ lower latency than the leading CKKS-based solution [CRYPTO'25]; (c) Up to $65\times$ speedups over existing FHE-ISA implementations, and $3\times \sim 40\times$ faster than related FHE compilers.

*Corresponding Author

†This work was initiated when Binwu Xiang was a Ph.D. student at IIE, CAS.

‡This work was completed during Hongyu Wang's internship at IIE, CAS.

1 Introduction

Fully Homomorphic Encryption (FHE) enables arbitrary computation on encrypted data while preserving confidentiality [65, 33]. It guarantees both the correctness of outputs and the privacy of inputs, making it a promising technology for secure computation. The characteristic of FHE critical advantages in high-stakes domains including finance, healthcare, and secure communications. Over the past decade, popular FHE schemes, including BGV/BFV [11, 10, 31], CKKS [17], and FHEW/TFHE [2, 30, 20, 21, 22] schemes have evolved significantly. These schemes have been deployed in privacy-preserving machine learning [45, 56, 46, 44, 28, 43], private information retrieval [3, 61, 32, 51, 73], private set intersection [16, 15, 25], and oblivious message retrieval [53, 54, 48], with performance gradually attaining practical viability.

Despite these advances, deploying FHE in real-world applications remains challenging. Although it supports arbitrary computation in theory, the randomness introduced by encryption causes a mismatch between encrypted and plaintext programming models. Current algorithms often require domain experts for correct interpretation, lacking systematic translation mechanisms for engineering applications. Thus, automated frameworks for compiling encrypted computation are essential to bridge the “last mile” from theory to practice.

FHE Compiler. To close this gap, FHE compilers have emerged as a key focus of both academia [38, 12, 19, 27, 4, 26, 68, 50, 49, 18, 8, 40, 7] and industry [35, 36, 62, 74]. It translates high-level programming languages into encrypted programs, serving as the cornerstone of the FHE ecosystem. However, most existing FHE compilers directly translate applications into homomorphic operators. This design tightly couples both traditional responsibilities and FHE tasks—such as noise evaluation, parameter selection, and plaintext space design—to the compiler front-end, introducing two systemic weaknesses: (a) *Optimization limitations*: Since the performance of homomorphic operators is highly sensitive to noise growth and execution context, optimizer strategies struggle to generalize across schemes and tasks; (b) *architecture incompatibility*: Operator developments should feed back to the front-end, forcing the compiler pipeline to undergo frequent restructuring—or large-scale redesigns—during algorithm updates, which undermines long-term stability, see Figure 1. These limitations restrict the performance and scalability of existing FHE compilers.

FHE-ISA. To address the above issues, one promising direction is the design of an FHE Instruction Set Architecture (FHE-ISA) [39, 59, 37, 67, 69]. The key idea is to introduce an abstraction layer between the compiler front-end and homomorphic operators, decoupling complex front-end responsibilities and providing a stable interface. The front-end can then leverage established optimization engines to target the ISA, while the operator layer uses the latest FHE developments to optimize instruction execution, thereby offloading FHE tasks. This opens the opportunity to abstract a standardized FHE-ISA with a uniform instruction format, establishing a consistent interface between compilers and FHE algorithms. Currently, efficient FHE-ISA designs typically implement instructions as truth tables and evaluate them using FHE look-up tables

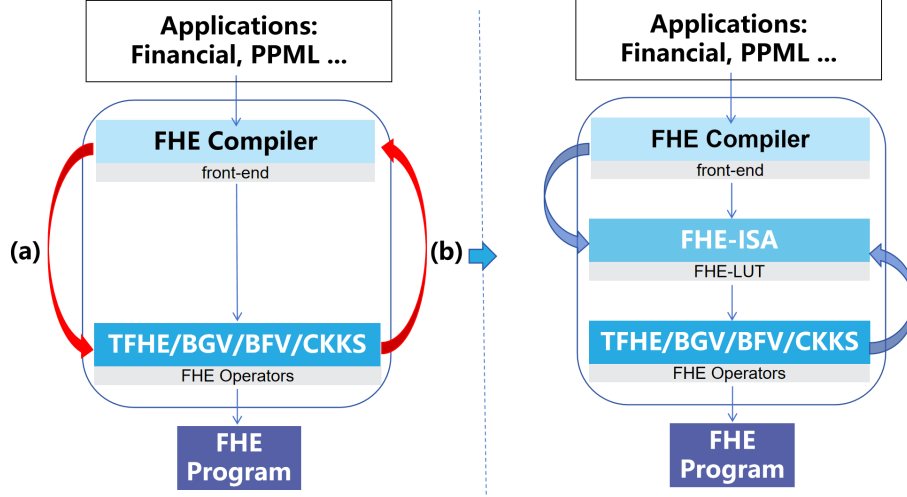


Figure 1: Existing FHE compiler and FHE-ISA paradigms. Labels (a) and (b) indicate the weaknesses caused by the tight coupling of front-end functionality discussed in the text, whereas FHE-ISA demonstrates cross-layer decoupling.

(LUT) [39, 37, 67, 69].

Challenges. Despite its promise, constructing practical FHE compilers based on FHE-ISA faces precision challenges. High-precision instruction sets are indispensable for supporting complex applications. For instance, financial workloads often demand 32-bit precision [66, 60], whereas current FHE LUT implementations are limited to a maximum of 16 bits, falling short of application requirements.

A straightforward approach to achieving higher precision is to assemble multiple small LUTs into a larger one. However, under encrypted computation this incurs exponential costs in runtime, storage, and noise growth—potentially leading to decryption failure. Existing efficient FHE LUT methods fall into three categories: TFHE programmable bootstrapping (also known as functional bootstrapping, PBS) [9, 22, 23, 24, 6] and PBS-tree [41, 67]; TFHE circuit bootstrapping (CBS) + CMux-tree [21, 5, 71, 69]; and the CKKS-based PBS recently proposed in CRYPTO’25 [1]. Although CKKS firstly supports LUT, its data precision decides the homomorphic multiplication depth, causing its modulus size to grow exponentially with precision and degrading performance. Similarly, TFHE PBS faces coupling between LUT precision and ciphertext polynomial dimension; when decomposed into multiple tables via PBS-tree, the number of costly bootstrapping calls grows exponentially (Figure 2), limiting practical performance. The CMux-tree approach reduces the bootstrapping dependence and represents a viable path, but its noise accumulation grows rapidly with increasing LUT precision. As a result, existing methods are limited to 16-bit precision and suffer efficiency loss due to parameter inflation.

1.1 Our Contributions

We present *Tetris*, an efficient and high-precision TFHE look-up-table framework. Building on Tetris, we design *TetrISA*, the first CMux-tree based FHE-ISA implementation, supporting up to 32-bit precision with 20+ instructions. Together, these contributions represent a concrete step toward practical FHE compilers.

FHE LUT. Tetris achieves three major advances:

- **Higher Precision via decoupling and noise control.** Noise accumulation and parameter inflation have been the primary barriers to scaling CMux-tree approaches. We propose *Rank Decoupled Scheme Switching* (Algorithm 4), which removes ciphertext dimension dependencies across LUT steps, combined with refined per-step noise management.

These techniques extend the LUT precision limit from previous 16 bits to **32 bits**. Moreover, at lower precision, the same framework allows more compact parameter sets, thus improving efficiency.

- **Higher Efficiency via batched circuit bootstrapping.** Under 16-bit precision, circuit bootstrapping dominates runtime. Beyond parameter-level gains, we improve *batched CBS* by constructing *bit-extraction function clusters* with large common factors. It halves the residual terms of each function and thereby doubling effective batching.

As a result, Tetris outperforms current TFHE LUT methods [6, 69, 67] by **2×-863×** at the same precision and achieves up to **2,901×** lower latency¹ compared with CKKS-based LUT evaluation [1].

- **Adaptivity via parameter tuning and parallelism.** Once LUT precision exceeds 24 bits, the runtime bottleneck shifts from CBS to CMux evaluation. This motivates new parameter selection strategies, alongside exploiting multi-threaded CMux-tree execution.

Tetris is the first FHE LUT framework supporting adaptive parameter tuning and parallel execution. It automatically selects parameter sets, encoding strategies, and LUT evaluation structures based on target precision and parallelism. Experiments show that, at 32-bit precision with 8 threads, our adaptive tuning yields a **9.5×** speedup over baseline configurations.

FHE-ISA. On top of these advances, we construct *TetrISA*, a high-precision and high-performance FHE instruction set:

- **Functionality.** TetrISA is the first FHE-ISA to support general 16-bit bivariate and 32-bit univariate operations, with **20+** commonly used instructions. It covers and extends the instruction sets of recent FHE-ISA designs [67, 69].
- **Performance.** Compared with the state-of-the-art FHE-ISA implementation [67], Tetris achieves up to **65×** performance improvement at the same precision. Against representative FHE compilers [19, 38, 74, 12], TetrISA delivers **3×-40×** faster execution on benchmark programs, and accelerates the 32-bit Sign evaluation by **2,916×** relative to CKKS-based solution [1].

¹Tetris throughput is $\sim 23\times$ lower than CKKS, since CKKS can pack 65,536 messages per ciphertext.

- **Practice.** A naive n -bit bivariate instruction requires a 2^{2n} LUT, making high-precision infeasible. We embed LUT pruning method into Tetris, which removes redundant branches in the CMux-tree. This (a) improves the efficiency of supported but costly instructions, and (b) makes 32-bit bivariate operations tractable.

As a proof of concept, we design a pruned comparator (EQ/GT/GTE/LT/LTE). Our method achieves up to $841\times$ speedup compared with baseline settings, enabling 32-bit bivariate comparisons to complete within 2.3 s.

2 Preliminaries of TFHE

In this section, we present the cryptography foundations of Tetris, focusing on the underlying TFHE scheme, ciphertext forms, and core building blocks of Tetris.

2.1 Notations

Throughout the paper, bold letters denote vectors (or matrices). The nearest integer to r is denoted $\lfloor r \rfloor$. For real numbers a and b such that $a < b$, we write $[a, b[= \{x \in \mathbb{R} : a \leq x < b\}$. For two integers a and b , $\mathbb{Z} \cap [a, b[$ is denoted $\llbracket a, b \rrbracket$. For an integer q , we identify $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ with $\llbracket -q/2, q/2 \rrbracket$, and $[\cdot]_q$ denotes the mod q reduction into $\mathbb{Z}_q$. The set $\mathbb{B}$ and $[n]$ denote $\{0, 1\}$ and $\{1, 2, \dots, n\}$, respectively, for a positive integer n . The set $\mathbb{N}$ denotes the set of all positive integers. The set $\mathbb{Z}_q^\times$ denotes the multiplicative subgroup of $\mathbb{Z}_q$. For a power-of-two N , the cyclotomic ring $\mathbb{Z}[X]/(X^N + 1)$ is denoted by $\mathcal{R}_N[X]$. We also write $\mathcal{R}_{q,N} = \mathcal{R}/q\mathcal{R} = \mathbb{Z}_q[X]/(X^N + 1)$ and $\mathbb{B}_N[X] = \mathbb{B}[X]/(X^N + 1)$. For a probability distribution $\mathcal{D}$, $a \leftarrow \mathcal{D}$ denotes that a is sampled according to the distribution $\mathcal{D}$. Unless stated otherwise, all logarithms are to the base 2.

2.2 Ciphertext Form

We briefly review the ciphertext forms of TFHE since Tetris switches between them as needed. We use p and q to denote the modulus of messages and ciphertexts, respectively.

LWE, RLWE, and GLWE. Under a secret key $\mathbf{S} \in \mathcal{R}_{q,N}^k$, a message $M \in \mathcal{R}_{p,N}$ is encrypted into a Generalized LWE (GLWE) ciphertext $\mathbf{C} \in \mathcal{R}_{q,N}^{k+1}$ with a scaling factor Δ such that $\Delta \leq q/p$ as follows [11].

$$\mathbf{C} = \text{GLWE}_{q,\mathbf{S}}(\Delta \cdot M) = (A_1, \dots, A_k, B = \sum_{i=1}^k A_i \cdot S_i + [M \cdot \Delta]_q + E)$$

where $\mathbf{S} = (S_1, \dots, S_k)$, $A_i \leftarrow \mathcal{R}_{q,N}$ for $i = 1, 2, \dots, k$, and $E \leftarrow \chi_\sigma$ for some Gaussian distribution χ_σ as the error distribution. kN is called the GLWE dimension and k is called the GLWE rank. It is common to use the binary secret key in TFHE implementation, and in this paper we only consider this

case. Some of subscripts $q, \mathbf{S}$ are omitted when they are clear from the context. For simplicity, B is sometimes denoted by A_{k+1} .

If $A_i = 0$ for $i = 1, 2, \dots, k$ and $B = [\Delta \cdot M]_q$, it is called a *trivial* GLWE ciphertext. It does not provide security for the plaintext M , but only encodes M in GLWE form.

A GLWE ciphertext with $N = 1$ is called an LWE ciphertext. In this case, it is common to use n to denote the LWE dimension instead of k for rank, so that an LWE ciphertext is usually denoted $(a_1, \dots, a_n, b) \in \mathbb{Z}_q^{n+1}$. When $k = 1$, a GLWE ciphertext is called a Ring LWE (RLWE) ciphertext.

The decryption of a GLWE ciphertext is to compute its *phase*, which is defined as $B - \langle (A_1, \dots, A_k), \mathbf{S} \rangle$, followed by rounding the phase by the scaling factor Δ . The decryption is correct if the error contained in the ciphertext is small enough to be removed by the rounding with Δ .

From the definition of the GLWE ciphertext, the sum of GLWE ciphertexts under the same secret key results in the sum of plaintexts in $\mathcal{R}_{p,N}$, with the error increasing linearly.

GLev. Let $B \in \mathbb{N}$ be a power-of-two and $\ell \in \mathbb{N}$. A GLev ciphertext $\overline{\mathbf{C}} \in \mathcal{R}_{q,N}^{(k+1)\ell}$ of $M \in \mathcal{R}_{q,N}$ with gadget length ℓ and base B under a GLWE secret key $\mathbf{S} \in \mathbb{B}_N[X]^k$ is defined as a vector of ℓ GLWE ciphertexts of $M \in \mathcal{R}_{q,N}$ as follows.

$$\overline{\mathbf{C}} = \text{GLev}_{\mathbf{S}}^{(B,\ell)}(M) = (\text{GLWE}_{\mathbf{S}}(v_j \cdot M))_{j \in [\ell]}$$

where $v_j = \lceil q/B^j \rceil$ for $j = 1, \dots, \ell$.

GGSW. For nonlinear operations such as multiplication, TFHE uses another type of ciphertext called generalized GSW (GGSW) [34]. Let $B \in \mathbb{N}$ be a power-of-two and $\ell \in \mathbb{N}$. A GGSW ciphertext $\overline{\overline{\mathbf{C}}} \in \mathcal{R}_{q,N}^{(k+1)\ell \times (k+1)}$ of a message $M \in \mathcal{R}_{q,N}$ with gadget length ℓ and base B under a secret key $\mathbf{S} \in \mathbb{B}_N[X]^k$ is an $(k+1)\ell \times (k+1)$ matrix over $\mathcal{R}_{q,N}$ defined as follows.

$$\overline{\overline{\mathbf{C}}} = \text{GGSW}_{\mathbf{S}}^{(B,\ell)}(M) = (\text{GLWE}_{\mathbf{S}}(v_j \cdot (-S_i \cdot M)))_{(i,j) \in [k+1] \times [\ell]}$$

where $v_j = \lceil q/B^j \rceil$ for $j = 1, \dots, \ell$, $\mathbf{S} = (S_1, \dots, S_k)$, $S_{k+1} = -1$. One can also represent $\overline{\overline{\mathbf{C}}}$ as a vector of $k+1$ GLev ciphertexts $(\text{GLev}_{\mathbf{S}}^{(B,\ell)}(-S_i \cdot M))_{i=1}^{k+1}$.

2.3 Building Blocks

We then introduce the building blocks of TFHE scheme and Tetris. We indicate some of the inputs and outputs for clarity.

Gadget Decomposition. Let $B \in \mathbb{N}$ be a power-of-two and $\ell \in \mathbb{N}$. The gadget decomposition $\text{GadgetDecomp}^{(B,\ell)}$ with a base B and length ℓ decomposes an input $a \in \mathbb{Z}_q$ into a vector $(a_1, \dots, a_\ell) \in \mathbb{Z}_q^\ell$ such that

$$a = \sum_{j=1}^{\ell} a_j \cdot v_j + e$$

where $v_j = \lceil q/B^j \rceil$, $a_j \in \llbracket -B/2, B/2 \rrbracket$ for all $j = 1, \dots, \ell$ and the decomposition error e satisfies $|e| \leq \lceil \frac{q}{2B^\ell} \rceil$. The gadget decomposition can be extended to a polynomial in $\mathcal{R}_{q,N}$ by applying the decomposition to its coefficients.

External Product. $(\text{GGSW}(M_1), \text{GLWE}(M_2)) \rightarrow \text{GLWE}(M_1 \cdot M_2)$. The external product $\boxtimes$ between a GGSW ciphertext $\overline{\mathbf{C}}_1$ and a GLWE ciphertext $\mathbf{C}_2$ is defined as

$$\overline{\mathbf{C}}_1 \boxtimes \mathbf{C}_2 = \sum_{i=1}^{k+1} \sum_{j=1}^{\ell} A_{i,j} \cdot \text{GLWE}(v_j \cdot (-S_i \cdot M_1))$$

where $\text{GLWE}(v_j \cdot (-S_i \cdot M))$ is each GLWE component of $\overline{\mathbf{C}}_1$ for $(i, j) \in [k+1] \times [\ell]$, $(A_{i,1}, \dots, A_{i,\ell})$ is a gadget decomposition of A_i for $i \in [k+1]$ such that $\mathbf{C}_2 = (A_1, \dots, A_{k+1})$, and the multiplication between a polynomial and a GLWE ciphertext denotes multiplying the polynomial to each polynomial component of the GLWE ciphertext. Then the result becomes a GLWE ciphertext of $M_1 M_2$.

CMux Gate. $(\text{GGSW}(b), \text{GLWE}(M_1), \text{GLWE}(M_2)) \rightarrow \text{GLWE}(M_b)$. The controlled mux gate, dubbed CMux, is the key operation used in TFHE. Suppose that two GLWE ciphertexts $\mathbf{C}_0$ and $\mathbf{C}_1$ are given along with a secret boolean value b encrypted to a GGSW ciphertext $\overline{\mathbf{C}}$, where all three ciphertexts are encrypted with the same key $\mathbf{S}$. Then one may select $\mathbf{C}_b$ without knowing b by

$$\text{CMux}(\overline{\mathbf{C}}, \mathbf{C}_0, \mathbf{C}_1) = (\mathbf{C}_1 - \mathbf{C}_0) \boxtimes \overline{\mathbf{C}} + \mathbf{C}_0.$$

Blind Rotation. $((\mathbf{C}_f, \text{LWE}(\Delta m), \text{BSK}) \rightarrow \text{GLWE}(f(m) + \dots))$. The blind rotation (BlindRotate) algorithm refreshes the ciphertext noise and evaluates a function for free. Suppose that an LWE ciphertext $\mathbf{c} = (a_1, \dots, a_n, b) \in \mathbb{Z}_q^{n+1}$ of a phase $\mu = \Delta m + e$ under a secret key $\mathbf{s} = (s_1, \dots, s_n) \in \mathbb{B}^n$ is given. The blind rotation algorithm outputs a refreshed GLWE ciphertext of a polynomial that $f(m)$ lies in its constant term (other terms are redundant), under a secret key $\mathbf{S}' \in \mathbb{B}^{kN}$ by following steps:

Encode the function f on a new GLWE ciphertext $\mathbf{C}_f$. The half of the function values of f are redundantly encoded in the coefficients of the plaintext of the (trivial) GLWE ciphertext.

Multiply $X^{-\tilde{b} + \sum_{i=1}^n \tilde{a}_i s_i} = X^{-\tilde{\mu}}$ to the GLWE ciphertext encoding the function using a bootstrapping key (BSK) $\{\text{GGSW}_{\mathbf{S}'}(s_i)\}_{i=1}^n$; multiply either 1 or $X^{-\tilde{a}_i}$ according to $s_i \in \{0, 1\}$ by the CMux gate.

Sample Extraction. The sample extraction SampleExtract_i extracts the i -th term of the GLWE encrypted polynomial, and outputs a corresponding LWE ciphertext. It extracts the constant term if i is omitted in this paper.

Key Switching, Automorphism and Bootstrapping. TFHE key switching, automorphism and bootstrapping (concluding PBS, PBSmanyLUT, MV-PBS, CBS) algorithms are combinations of the aforementioned building blocks. We detail them in Appendix A for self completeness.

3 Revisiting FHE Look-Up-Table

In this section, we revisit FHE-based LUT techniques. The last systematic discussion appeared in JoC’23 [5], but since then several new approaches have been proposed and improved. We summarize recent progress and provide a comparative discussion. Detailed experimental results are presented in Section 5.2.

Programmable Bootstrapping (PBS) [9, 22, 23, 24, 6]. The most direct LUT evaluation in TFHE is via programmable bootstrapping. As described in the Blind Rotation algorithm, one can embed either semi-domain functions ($f : N \rightarrow 2N$) or negacyclic functions ($f : 2N \rightarrow 2N, f(N + i) = -f(i)$). However, the achievable precision is tightly bounded by the polynomial degree N of the cyclotomic ring (see Appendix C for details). Increasing precision requires enlarging N , which leads to a severe performance overhead. A recent advance by Bergerat et al. [6], introduces a new secret key format with sparse keys and shared randomness. This approach reduces noise growth and thus moderately improves message precision. However, the constraint between N and precision remains, and our experiments show PBS becomes impractical beyond 7-bit precision, see Figure 4.

PBS Tree [41, 67]. To mitigate parameter blow-up, Guimarães et al. [41] introduced a tree-based bootstrapping method (referred to PBS-tree in this paper). The ciphertext is partitioned into $\lceil \delta/\tau \rceil$ blocks of precision τ , and bootstrapped in a tree structure (Figure 2(a)). The cost arises from the exponential number of PBS calls, $\sum_{i=0}^{\lceil \delta/\tau \rceil - 1} 2^{i\tau}$, and from the *Repacking* step, which is even slower than a PBS [67]. A later development [67] replaces all PBS in the first layer with a single Multi-Value PBS (MV-PBS) [13], reducing the number of PBS calls to $\sum_{i=0}^{\lceil \delta/\tau \rceil - 2} 2^{i\tau} + 1$. However, as the number of bootstrappings still grows exponentially, PBS-tree becomes significantly slower than the following CMux-tree method for high-precision LUT evaluations.

CMux-Tree [21, 5, 71, 69]. The main bottleneck of PBS-tree is that every leaf node requires a PBS, each costing $O(n)$ CMux gates. Chillotti et al. [21] proposed replacing these with direct CMux gates: each leaf requires only one CMux. However, this process accumulates noise, and CMux takes GGSW ciphertexts as input but outputs LWE ciphertexts. So that it must be refreshed and converted back into GGSW form via *circuit bootstrapping* (Figure 2(b)) before next circuit evaluation. Thus, CMux-tree consists of three steps: refreshing, ciphertext conversion, and CMux evaluation, with the first two together forming CBS. The key advantage of CMux-tree method is that bootstrapping is needed only once per circuit input, so the number of bootstrappings grows linearly (not exponentially) with precision. This yields better efficiency and scalability of TFHE LUT [5, 71]. The most recent work by Wang et al. proposes integer-CBS[69], which refreshes and converts two bits simultaneously and outperforms PBS and PBS-tree beyond 7-bit precision (see Section 5.2). Building on this line of research, Tetris extends and improves the approach, consistently surpassing the state-of-the-art across all tested precision levels.

Interpolation Polynomials [55, 52, 1]. An alternative direction is to evalu-

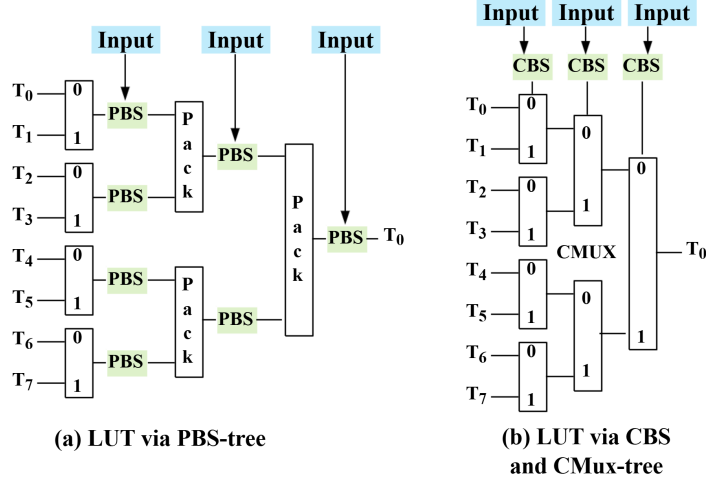


Figure 2: PBS-tree v.s. CMux-tree

ate LUTs with interpolation polynomials. Liu et al. [55] used polynomial interpolation to simulate rounding and target LUT functions in BGV/BFV, enabling amortized bootstrapping. More recently, Alexandru et al. proposed a state-of-the-art CKKS-based LUT method [1]. To ensure accurate LUT evaluation in CKKS, they introduced trigonometric Hermite interpolation, which matches the LUT values and sets the first derivative zero at all interpolation points. For an LUT of size δ , this gives a degree- δ trigonometric polynomial, rewritten as a degree- δ power series in $e^{2\pi ix}$. The exponential is approximated via Chebyshev polynomials, and evaluation uses the Paterson–Stockmeyer algorithm [63]. This method yields the current best performance in interpolation polynomial category, and serves as the main CKKS baseline for our comparisons.

4 Tetris: A Versatile TFHE LUT Framework

In this section, we introduce Tetris, which follows the CBS+CMux-tree paradigm and is structured into three core steps: *refreshing*, *ciphertext conversion*, and *CMux evaluation*. Before detailing each step, we provide a high-level overview of techniques.

Decoupled Parameters with Refined Noise Control. Existing TFHE LUT frameworks can be summarized as Figure 3:

- *Staircase*: Introduced by the first circuit bootstrapping design [21] and later adopted in BBB+ (JoC’23) [5]. The refresh step uses the largest parameter set to absorb accumulated noise, followed by smaller parameters for later stages to optimize performance.
- *Rectangular*: Adopted in WWL+ (EUROCRYPT’24) [71]. This approach introduced the homomorphic trace function to reduce multiplication complexity

Algorithm 1: Tetris: A Versatile TFHE Look-Up-Table Framework

Input: $\mathbf{c}_i = \text{LWE}_{\mathbf{s}}(\Delta m_i)$ for $i = 0, \dots, \lceil \frac{\delta}{\tau} \rceil - 1$
Input: ℓ test polynomials F_k to compute $\frac{v_k}{2} \cdot (-1)^{m_{i,0}+1}$ for $k = 1, \dots, \ell$ from a τ -bit input $m_i = \sum_{j=0}^{\tau-1} m_{i,\tau-1-j} \cdot 2^j$
Input: $\tau - 1$ polynomials G_j such that $G_j \cdot F_k$ becomes a test polynomial to compute $\frac{v_k}{2} \cdot (-1)^{\tilde{m}_{i,j}+1}$ for $j = 1, \dots, \tau - 1$
Input: Bootstrapping keys under $\mathbf{S}$ for TetrisRefresh
Input: Automorphism keys under $\mathbf{S}$ for RevHomTrace
Input: Scheme switching key under $\mathbf{S}'$ for DecSchemeSwitch
Input: GLWE Packed LUT value $T_0, \dots, T_{\lceil \frac{\delta}{\tau} \rceil \cdot 2^\delta}$ for homomorphic LUT with mixed packing LUT
Output: $\mathbf{c}'_i = \text{LWE}_{\mathbf{s}}(\Delta f(m)_i)$ for $i = 0, \dots, \lceil \frac{\delta}{\tau} \rceil - 1$

```

1 for  $i = 0$  to  $\lceil \frac{\delta}{\tau} \rceil - 1$  do
2    $\mathbf{C}'_{i,j,k} \leftarrow \text{TetrisRefresh}(\mathbf{c}_i), j = 0, \dots, \tau - 1, k = 0, \dots, \ell - 1$ 
   /*  $\mathbf{C}'_{i,j,k} = \text{GLWE}_{\mathbf{s}}(\tilde{m}_{i,j} + \dots)$  */
3   for  $j = 0$  to  $\tau - 1$  do
4     for  $k = 0$  to  $\ell - 1$  do
5        $\mathbf{C}'_{i,j,k} \leftarrow \text{RevHomTrace}(\mathbf{C}_{i,j,k})$  /*  $\mathbf{C}'_{i,j,k} = \text{GLWE}_{\mathbf{s}}(\tilde{m}_{i,j})$  */
6        $\overline{\mathbf{C}}'_{i,j}[k] = \mathbf{C}'_{i,j,k}$ 
7      $\overline{\mathbf{C}}'_{i,j} \leftarrow \text{DecSchemeSwitch}(\overline{\mathbf{C}}'_{i,j})$  /*  $\overline{\mathbf{C}}'_{i,j} = \text{GGSW}_{\mathbf{S}'}(\tilde{m}_{i,j})$  */
8    $\mathbf{c}'_i \leftarrow \text{LUT}(\overline{\mathbf{C}}'_{i,j})$  /*  $\mathbf{c}'_i = \text{LWE}_{\mathbf{S}'}(\Delta f(m)_i)$  */
9    $\mathbf{c}'_i \leftarrow \text{SR} - \text{KeySwitch}(\mathbf{c}'_i)$  /*  $\mathbf{c}'_i = \text{LWE}_{\mathbf{S}'}(\Delta f(m)_i)$  */
10   $\mathbf{c}'_i \leftarrow \text{MC} - \text{ModSwitch}(\mathbf{c}'_i)$  /*  $\mathbf{c}'_i = \text{LWE}_{\mathbf{s}}(\Delta f(m)_i)$  */
11 return  $\mathbf{c}'_i$ 

```

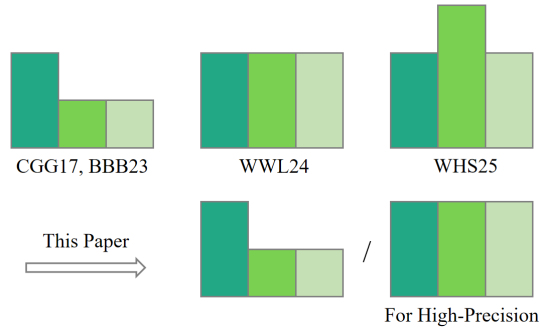


Figure 3: CMux-tree based LUT framework according to parameter sizes across different steps. Taller shapes indicate larger parameters.

Table 1: Theoretical Comparison of TFHE LUT Methods

Solution	Refresh	Conversion	Evaluation
PBS [9, 22, 23, 24, 6]	1 Bootstrapping with Exponential Dimension		
PBS-Tree [41]		$\sum_{i=0}^{\lceil \frac{\delta}{\tau} \rceil - 1} 2^{i\tau}$ PBS	
PBS-Tree [67]		$\sum_{i=0}^{\lceil \frac{\delta}{\tau} \rceil - 2} 2^{i\tau} + 1$ PBS	
CMux-tree [21, 5]	$\delta(\ell + 1)$ PBS	Private KS	$\approx \frac{2^\delta}{N}$ CMux
CMux-tree [71]	δ PBSmanyLUT	HomTrace, SS	$\approx \frac{2^\delta}{N}$ CMux
CMux-tree [69]	$\lceil \frac{\delta}{2} \rceil$ PBSmanyLUT	HP-HomTrace, SS	$\approx \frac{2^{\delta-1}}{N}$ CMux
CMux-tree [Ours]	$\lceil \frac{\delta}{\tau} \rceil$ PBSmanyLUT	RevHomTrace, DecSS	$\approx \frac{2^\delta}{\tau N}$ CMux

(a) Except for PBS, all methods rely on PBS calls with constant dimension. While the conversion step costs are similar across schemes (except for the weaker Private KS in [21, 5], see analysis in WWL24 [71]), the key advantage of Tetris lies in RevHomTrace and DecSS, which decouple stage parameters and enable exponential CMux evaluations under smaller parameters.

in the conversion step, but introduced parameter coupling, as all stages must share the same polynomial ring dimension.

- *Convex*: Proposed in WHS+ (CCS’25) [69], enabling integer circuit bootstrapping. However, due to excessive noise growth in high-precision HomTrace evaluation, the conversion step requires larger parameters.

Tetris introduces a versatile LUT framework that addresses flexibility and noise issues of prior works. Specifically:

- Tetris adopts the general GLWE ciphertext format and design a *decoupled scheme switching algorithm* (Algorithm 4), which breaks the parameter dependency between refreshing and conversion steps.
- Tetris introduce refined noise management techniques²:
 - (a) Conversion step: *Amplification removing reverse homomorphic trace evaluation* (Algorithm 3);
 - (b) Evaluation step: *Shared randomness LWE key switching* (Algorithm 5) and *mean compensation modulus switching* (Algorithm 6).

Together, these techniques enable tighter parameterization—supporting higher precision under the same parameters, or achieving more efficient evaluation at the same precision.

Batched Circuit Bootstrapping. Batched bootstrapping in TFHE is a challenging topic. PBS supports two single-input, multi-output techniques: (1) *MV-PBS*, which evaluates a common factor f_0 from a target function set $\{f_i\}_{i=1}^k$ using PBS, and then computes all outputs by multiplying $\frac{f_i}{f_0}$ [13]; (2) *PBSmanyLUT*, which clears ϑ noise bits using modulus switching before bootstrapping, enabling encoding of up to 2^ϑ functions in one PBS [24]. In the CBS+CMux-tree

²We further explain why Tetris benefits from noise management for clarity. This additional discussion is included in Appendix C.

The main bottleneck of this batched method also lies in noise management. In MV-PBS, the norm of the post-multiplication $\frac{f_i}{f_0}$ grows exponentially with the batch factor τ , resulting in a noise blow-up of $2^\tau - 1$. We lower this bound by redesigning the function family and introducing a new batched refreshing algorithm, **TetrisRefresh** (Algorithm 2). The algorithm is equipped with an optimized noise-tracing (Appendix B), which extends the batching capacity of CBS and significantly improves overall system performance.

Algorithm 2: TetrisRefresh, combining PBSmanyLUT and MV-PBS with newly designed negacyclic function cluster

4.1 Refresh Step

12

Variable i indexes messages, $i \in \{0, \dots, \lceil \frac{\delta}{\tau} \rceil - 1\}$. Variable j indexes bits in each message, $j \in \{0, \dots, \tau - 1\}$. Variable k indexes gadget decomposition length ℓ of GLev/GGSW ciphertexts, with decomposition vector $\{v_k\}, k = 1, \dots, \ell$.

Refresh step. The input consists of $\lceil \frac{\delta}{\tau} \rceil$ LWE ciphertexts $\text{LWE}_s(\Delta m_i)$, each encrypting a τ -bit message $m_i = \sum_{j=0}^{\tau-1} m_{i,\tau-1-j} \cdot 2^j$. Here, $m_{i,0}, \dots, m_{i,\tau-1}$ denote the bits of m_i from most to least significant. The output comprises δ GLev ciphertexts, each encrypting a single-bit message obtained after bootstrapping refresh: $\text{GLew}_s(\tilde{m}_{i,0} + \dots), \dots, \text{GLew}_s(\tilde{m}_{i,\tau-1} + \dots)$, where $\tilde{m}_{i,0} = m_{i,0}$ and $\tilde{m}_{i,j} = m_{i,0} \oplus \dots \oplus m_{i,j}$ for $j \geq 1$. The purpose of this step is to refresh the ciphertext noise and to perform necessary pre-processing.

Newly Designed Negacyclic Function. The ideal output of the refresh step would be $\text{GLew}_s(m_{i,0} + \dots), \dots, \text{GLew}_s(m_{i,\tau-1} + \dots)$. However, for $j \neq 0$, the bit-extraction function $f : m_i \mapsto m_{i,j}$ does not satisfy the negacyclic property required for PBS evaluation (see Section 3 for details). To resolve this, we introduce a new function family $f : m_i \rightarrow \tilde{m}_{i,j}$, where $\tilde{m}_{i,0} = m_{i,0}$ and $\tilde{m}_{i,j} = m_{i,0} \oplus \dots \oplus m_{i,j}$ for $j \geq 1$. The LUT design in the evaluation step is accordingly modified to ensure correctness. The concrete computation for the refresh step is given below.

Combine PBSmanyLUT with MV-PBS. The key idea of batched CBS is to combine PBSmanyLUT with MV-PBS, enabling multiple messages to be refreshed and precomputed within a single bootstrapping. Our proposed algorithm, TetrisRefresh, is detailed in Algorithm 2. Concretely, PBSmanyLUT is first used to extract and compute redundant GLWE ciphertexts for the most significant bit: $\text{LWE}_s(\Delta m_i) \rightarrow \text{GLew}_s(\tilde{m}_{i,0} + \dots) = \{\text{GLWE}_s(v_k \cdot \tilde{m}_{i,0} + \dots)\}_{k \in \{0, \dots, \ell-1\}}$, where v_k are gadget basis vectors. For each message block m_i , the refresh computation proceeds identically. We then omit the block subscript i in the following descriptions for brevity.

For a ciphertext $\text{LWE}(m)$, extracting the most significant bit m_0 and producing the k GLev components requires PBSmanyLUT to evaluate the following k functions: $F_{m_0,k}(X) = -\frac{v_k}{2} (1 + X + \dots + X^{N-1})$. The blind rotation output is $\text{GLWE}(\frac{v_k}{2} \cdot (-1)^{m_0+1} + \dots)$, and by adding a fresh $\text{GLWE}(\frac{v_k}{2})$, one obtains $\text{GLWE}(v_k \cdot m_0 + \dots)$, as shown in line 1 and line 6 of Algorithm 2.

Next, MV-PBS is used to generate the remaining bits. For the second bit (counted from the MSB), we evaluate its XOR with the MSB, noting that

$$F_{m_1 \oplus m_0}(X) = \sum_{k=0}^{\frac{N}{2}-1} X^k - \sum_{k=\frac{N}{2}}^{N-1} X^k = -X^{N/2} \cdot F_{m_0}(X)$$

Therefore, $f_{m_1 \oplus m_0}$ can be evaluated on the same input by multiplying $-X^{N/2}$ to the blind rotation output on $f_{m_0,k}$ in the MV-PBS framework. More generally, for the j -th bit we compute $G_j(X) \cdot F_{m_0}(X)$ using MV-PBS, where $G_j(X)$ contains $(2^{j-1} + 1)$ terms and is defined as

$$G_j(X) = \begin{cases} 1 + \sum_{\substack{1 \leq r \leq 2^j - 1 \\ r \text{ odd}}} (-1)^{\frac{r+1}{2}} X^{\frac{r}{2}}, & j \text{ even,} \\ -X^{1/2} + \sum_{\substack{1 \leq r \leq 2^j - 1 \\ r \text{ odd}}} (-1)^{\lfloor \frac{r+1}{2} \rfloor} X^{\frac{r}{2}}, & j \text{ odd,} \end{cases}$$

The construction corresponds to lines 2-5 of Algorithm 2.

Remark 1 (Comparison with WHS+). *WHS+ [69] employs the function family $f : m_i \mapsto m_{i,0} \oplus m_{i,j}$, where the auxiliary polynomial G_j in MV-PBS contains $2^j - 1$ terms. This design leads to a large multiplier norm and higher noise growth. In contrast, we redesign the functions to improve the precision of batched refreshing bits. Our construction reduces the norm of the auxiliary multiplier by roughly a factor of two and decreases the number of terms in G_j to $2^{j-1} + 1$, thereby mitigating noise accumulation. Furthermore, we correct the noise estimation procedure of WHS+; details are provided in Appendix B on noise analysis.*

4.2 Conversion Step

Conversion Step. The input is a ciphertext of the form $\text{GLev}_{\mathbf{S}}(\tilde{m}_{i,j} + \dots)$, and the output is $\text{GGSW}_{\mathbf{S}'}(\tilde{m}_{i,j})$. The purpose of this step is to eliminate redundant terms in the plaintext polynomial and convert the ciphertext form.

Amplification Removing Reverse HomTrace [47]. A standard approach to eliminate redundant powers of X is to homomorphically evaluate the trace function, leveraging its favorable properties over finite fields:

$$\text{Tr}(\mu(X)) := \sum_{d \in \mathbb{Z}_{2^N}^\times} \tau_d(\mu(X)) = N\mu_0.$$

There are three variants in this step: (a) HomTrace with pre-processing [71], which cancels the amplification factor N on the constant term by multiplying N^{-1} in the NTT setting (or $\frac{1}{N}$ in the FFT setting). However, this approach suffers from significant noise amplification; (b) High Precision HomTrace (HP-HomTrace) [69], which performs GLWE rank switching so that the trace is evaluated at larger parameters, thereby accommodating higher noise but at the cost of inducing a convex circuit-bootstrapping structure; and, most recently, (c) Amplification Removing Reverse HomTrace (RevHomTrace) algorithm of Yoon et al. [47], which removes the amplification factor at each iteration, thereby avoiding factor- N amplification in error accumulation.

In conversion step, we adopt the method of RevHomTrace, implementing $\text{GLev}_{\mathbf{S}}(\tilde{m}_{i,j} + \dots) \xrightarrow{\text{RevHomTrace}} \text{GLev}_{\mathbf{S}}(\tilde{m}_{i,j})$. The key advantage is that the noise accumulation of the homomorphic trace is reduced from $O(N^3)$ to $O(N \log N)$, as claimed in Theorem 4 from [47].

Algorithm 3: Amplification Removing Reverse Homomorphic Trace Evaluation (RevHomTrace [47])

Input: $\mathbf{C} = \text{GLWE}_{\mathbf{S}(X)}(M(X))$ where
 $M(X) = m_0 + m_1X + \dots + m_{N-1}X^{N-1}$
Input: Automorphism key $\text{AutoKey}_d = \text{KS}_{\mathbf{S}(X^d) \rightarrow \mathbf{S}(X)}$ for all $d \in \mathbb{Z}_{2N}^\times$
Output: $\mathbf{C}' = \text{GLWE}_{\mathbf{S}(X)}(m_0)$

```

1  $\mathbf{C}' \leftarrow \mathbf{C}$ 
2 for  $d = 1$  to  $\log(N)$  do
3    $\mathbf{C}' = (A'_1, \dots, A'_{k+1})$ 
4    $\mathbf{C}' \leftarrow (\lfloor \frac{A'_1}{2} \rfloor, \dots, \lfloor \frac{A'_{k+1}}{2} \rfloor)$ 
5    $\mathbf{C}' \leftarrow \mathbf{C}' + \text{EvalAuto}(\mathbf{C}', 2^d + 1)$ 
6 return  $\mathbf{C}'$ 

```

Rank Decoupled Scheme Switching. The output of RevHomTrace is $\text{GLev}_{\mathbf{S}}(\tilde{m}_{i,j})$, which should be converted into GGSW ciphertexts. In traditional approach, this is done by first computing $\text{GLev}_{\mathbf{S}}(\tilde{m}_{i,j}S_t)$ for $t \in \{1, \dots, k\}$, where k is the GLWE rank, and then assembling these components together with $\text{GLev}_{\mathbf{S}}(\tilde{m}_{i,j})$ to obtain $\text{GGSW}_{\mathbf{S}}(\tilde{m}_{i,j})$ [71, 69, 72, 70].

However, the GGSW dimension kN significantly impacts the efficiency of the following CMux-tree evaluation. We thus propose a Rank Decoupled Scheme Switching (DecSchemeSwitch, Algorithm 4). By embedding rank switching into the traditional approach, DecSchemeSwitch transforms $\text{GLev}_{\mathbf{S}}(\tilde{m}_{i,j})$ into $\text{GGSW}_{\mathbf{S}'}(\tilde{m}_{i,j})$. Importantly, it allows the rank of the resulting GGSW ciphertext (i.e., the rank of $\mathbf{S}'$) to be smaller than k .

4.3 Evaluation Step

Evaluation Step. The input to this step is $\text{GGSW}_{\mathbf{S}'}(\tilde{m}_{i,j})$, and the output is $\text{LWE}_{\mathbf{S}}(f(m)_i)$, where $f(m)_i$ denotes the i -th τ -bit block of $f(m)$. This step performs the final CMux evaluation.

Homomorphic LUT with mixed packing. LUT evaluation from GGSW ciphertexts is a well-established technique, typically organized as a binary selection tree in which CMux gates—controlled by the GGSWs obtained in the previous step—select among GLWE ciphertexts. Our improvement follows the mixed packing method introduced by Chillotti et al. [20], leveraging the fact that a single GLWE ciphertext can encode N messages.

The procedure is as follows (see Figure 8 for a toy example). First, enumerate the truth table of the function. For a δ - δ LUT, the table contains $\lceil \frac{\delta}{\tau} \rceil \cdot 2^\delta$ entries³. Next, pack every N values into a GLWE ciphertext. Within each ciphertext, priority is given to the $\delta' = \lceil \frac{\delta}{\tau} \rceil$ entries in each row, and up to $\lfloor \frac{N}{\delta'} \rfloor$

³For simplicity, readers can assume exact divisions. In practice, parameters are always chosen as powers of two.

Algorithm 4: Rank Decoupled Scheme Switching (DecSchemeSwitch)

Input: $\overline{\mathbf{C}} = \text{GLew}_{\mathbf{S}}^{(B,\ell)}(M)$ under $\mathbf{S} = (S_1, \dots, S_k)$
Input: Scheme switching key $\text{SS}[i]_0 = \text{GLew}_{\mathbf{S}'}^{(B_{\text{ss}}, \ell_{\text{ss}})}(S_i)$ for $i = 1, \dots, k$;
 $\text{SS}[i]_1 = \text{GLew}_{\mathbf{S}'}^{(B_{\text{ss}}, \ell_{\text{ss}})}(S_i \sum_{t=1}^{k'} S_t)$ for $i = 1, \dots, k$ under
 $\mathbf{S}' = (S_1, \dots, S_{k'})$ where $k' \leq k$
Output: $\overline{\mathbf{C}'} = \text{GGSW}_{\mathbf{S}'}^{(B,\ell)}(M)$
1 $\overline{\mathbf{C}} = (\mathbf{C}_j)_{j \in [\ell]}$ where $\mathbf{C}_j = \text{GLWE}_{\mathbf{S}}(\frac{q}{B^j} \cdot M) = (A_{1,j}, \dots, A_{k,j}, A_{k+1,j})$
2 **for** $t = 1$ **to** k' **do**
3 **for** $j = 1$ **to** ℓ **do**
4 $\mathbf{C}'_{t,j} \leftarrow (0, \dots, 0, A_{k+1,j}, 0, \dots, 0)$, where $A_{k+1,j}$ at t -th term
5 **for** $i = 1$ **to** k **do**
6 $\mathbf{C}'_{t,j} \leftarrow \mathbf{C}'_{t,j} - \sum_{j_{\text{ss}}=1}^{\ell_{\text{ss}}} A'_{i,j,j_{\text{ss}}} \cdot \text{SS}[i]_1[j_{\text{ss}}]$
7 $\overline{\mathbf{C}}'_t = \{\mathbf{C}'_{t,j}\}_{j \in [\ell]}$ /* $\overline{\mathbf{C}}'_t = \text{GLew}_{\mathbf{S}'}(M \cdot \mathbf{S}_t)$ */
8 **for** $j = 1$ **to** ℓ **do**
9 **for** $i = 1$ **to** k **do**
10 Decompose $A_{i,j}$ as $\sum_{j_{\text{ss}}=1}^{\ell_{\text{ss}}} A'_{i,j,j_{\text{ss}}} \cdot \frac{q}{B_{\text{ss}}^{j_{\text{ss}}}} + E'_{i,j}$ with
 $\|A'_{i,j,j_{\text{ss}}}\|_{\infty} \leq \frac{B_{\text{ss}}}{2}$ and $\|E'_{i,j}\|_{\infty} \leq \frac{q}{2B_{\text{ss}}^{\ell_{\text{ss}}}}$
11 $\mathbf{C}'_j \leftarrow (0, \dots, 0, A_{k+1,j}) - \sum_{j_{\text{ss}}=1}^{\ell_{\text{ss}}} A'_{i,j,j_{\text{ss}}} \cdot \text{SS}[i]_0[j_{\text{ss}}]$
12 $\overline{\mathbf{C}}'_{k'+1} = \{\mathbf{C}'_j\}_{j \in [\ell]}$ /* $\overline{\mathbf{C}}'_{k'+1} = \text{GLew}_{\mathbf{S}'}(M)$ */
13 **return** $\overline{\mathbf{C}'} = (\overline{\mathbf{C}}'_t)_{t \in [k+1]}$ /* $\overline{\mathbf{C}'} = \text{GGSW}_{\mathbf{S}'}(M)$ */

rows are packed together. This yields $\lceil \frac{\delta' \cdot 2^\delta}{N} \rceil$ ciphertexts in total.

During evaluation, the first $\lceil \log(\lceil \frac{\delta' \cdot 2^\delta}{N} \rceil) \rceil$ GGSW inputs select the ciphertext containing the target output, requiring $\lceil \frac{\delta' \cdot 2^\delta}{N} \rceil - 1$ CMux gates. The remaining GGSW inputs is used to blind rotate the selected ciphertext so that the desired function value is shifted into the first δ' coefficients, requiring an additional $\log\lceil \frac{N}{\delta'} \rceil$ CMux gates. Finally, use [Sample Extraction](#) to produce the corresponding $\delta' = \lceil \frac{\delta}{\tau} \rceil$ LWE ciphertexts.

It is worth noting that Tetris applies the truth table mapping $\tilde{m}_{i,j} \mapsto f(m_{i,j})$ rather than $m_{i,j} \mapsto f(m_{i,j})$, ensuring consistency with the TetrisRefresh step.

Noise Control. The homomorphic LUT step takes $\text{GGSW}_{\mathbf{S}'}(\tilde{m}_{i,j})$ as input and outputs $\text{LWE}_{\mathbf{S}'}(\Delta f(m)_i)$. For clarity, we denote this ciphertext by $\text{LWE}_{Q,kN}(\Delta f(m)_i)$ according to its dimension and modulus. For performance considerations, we adjust the ciphertext parameters before blind rotation: an LWE-to-LWE key switching converts it to $\text{LWE}_{Q,n}(\Delta f(m)_i)$ (line 9, Algorithm 1), followed by modulus switching to $\text{LWE}_{2N,n}(\Delta f(m)_i)$ (line 10, Algorithm 1). Tetris applies two noise control techniques at this stage:

- Key Switching via Shared Randomness Secret Key [6]: The LWE-to-LWE key

Algorithm 5: LWE Key Switching via Shared Randomness Secret Key (SR-KeySwitch)

Input: $\mathbf{c} = \text{LWE}_{\mathbf{s}_1}(\Delta m)$ under $\mathbf{s}_1 = (s_1, \dots, s_{kN}) = \mathbf{s}_2 || \mathbf{s}_3$
Input: Key switching keys $\text{KSK}[i] = \text{GLew}_{\mathbf{s}_2}^{(B_{\text{ks}}, \ell_{\text{ks}})}(s_i)$ for $i = n + 1, \dots, kN$
Output: $\mathbf{c}' = \text{LWE}_{\mathbf{s}_2}(\Delta m)$
1 $\mathbf{c} = (a_1, \dots, a_{kN}, b)$
2 $\mathbf{c}' = (a_1, \dots, a_n, b)$
3 **for** $i = n + 1$ **to** kN **do**
4 Decompose a_i as $\sum_{j=1}^{\ell_{\text{ks}}} a_{i,j} \cdot \frac{q}{B_{\text{ks}}^j} + E_{i,j}$ $\mathbf{c}' = \mathbf{c}' + \sum_{j=0}^{\ell} a_{i,j} \cdot \text{KSK}[i][j]$
5 **return** $\mathbf{c}'$

switching procedure transforms a ciphertext under a key $\mathbf{s}_1 \in \mathbb{Z}^{kN}$ into one under a smaller key $\mathbf{s}_2 \in \mathbb{Z}^n$, where typically $n < kN$.

We define $\mathbf{s}_1$ such that it shares randomness with $\mathbf{s}_2$. Specifically, we set $\mathbf{s}_1 = \mathbf{s}_2 || \mathbf{s}_3$ where $\mathbf{s}_2 \in \mathbb{Z}^n$ forms the first n components of $\mathbf{s}_1$, and $\mathbf{s}_3 \in \mathbb{Z}^{kN-n}$ is randomly chosen. This structure, termed the *Shared Randomness Secret Key Problem*, can be reduced to a standard LWE problem [6].

Applying this structure to key switching reduces the evaluation key size, computational cost, and noise accumulated by a factor of $\frac{kN}{kN-n}$. Since it requires only $kN - n$ iterations instead of the full kN . The complete algorithm is formalized in Algorithm 5.

- **Modulus Switching via Mean Compensation [29]:** Modulus switching from Q to $2N$ is performed by scaling and rounding ciphertext coefficients: $\hat{a}_i = \lfloor (2N/Q) \cdot a_i \rfloor$ and $\hat{b} = \lfloor (2N/Q) \cdot b \rfloor$. This rounding introduces approximation errors $u_i = (2N/Q) \cdot a_i - \hat{a}_i$ and $u_b = (2N/Q) \cdot b - \hat{b}$. The induced noise term is $e_{ms} = \sum_{i=1}^n u_i s_i - u_b$ with variance $\text{Var}(e_{ms}) = \sigma_u^2(n(\sigma_s^2 + \mu_s^2) + 1)$ which depends on the secret's mean μ_s .

To remove this term, we apply *mean compensation*: a public correction term $\mu_s \sum u_i$ is subtracted from the ciphertext body before rounding. This yields an optimized noise term with variance $\sigma_u^2(n\sigma_s^2 + 1)$. The complete algorithm is formalized in Algorithm 6.

Tetris is obtained by composing the above steps. The complete procedure is detailed in Algorithm 1.

5 LUT Implementation

5.1 Security, Correctness and Parameters

Parameters for Security. The security of the TFHE scheme is based on the (general) learning with errors problem [64, 57]. The concrete security level is determined by the key distribution, ciphertext dimension, and initial noise, and

Algorithm 6: Modulus Switching with Mean Compensation (MC-ModSwitch)

Input: $\mathbf{c} = \text{LWE}_s(\Delta m)$ with the ciphertext modulus Q
Input: Mean of the secret key μ_s
Output: $\mathbf{c}' = \text{LWE}_s(\Delta m)$ with the ciphertext modulus $2N$

```

1  $\mathbf{c} = (a_1, \dots, a_n, b)$ 
2 for  $i = 1$  to  $n$  do
3    $a'_i = \left\lfloor \frac{2N}{Q} \cdot a_i \right\rfloor \bmod 2N$ 
4    $u_i = \frac{2N}{Q} \cdot a_i - a'_i$ 
5  $\text{comp} = \mu_s \cdot \sum_{i=1}^n u_i$ 
6  $b_{\text{comp}} = b - \frac{Q}{2N} \cdot \text{comp}$ 
7  $b' = \left\lfloor \frac{2N}{Q} \cdot b_{\text{comp}} \right\rfloor \bmod 2N$ 
8  $\mathbf{c}' = (a'_1, \dots, a'_n, b')$ 
9 return  $\mathbf{c}'$ 

```

Table 2: Parameters.

Name	Q	n	Std. Dev. LWE	N	k	Std. Dev. GLWE	ℓ_{ks}	B_{ks}	ℓ_{pbs}	B_{pbs}	ℓ_{tr}	B_{tr}	b_{tr}	ℓ_{ss}	B_{ss}	ℓ	B	ϑ	τ
Set I	2^{64}	710	2.8147×10^{14}	1024	1, 2	8.192×10^3	4	2^4	3	2^{12}	4	2^{10}	38	3	2^{13}	2	2^8	1	4
Set II	2^{64}	710	2.8147×10^{14}	1024	1, 2	8.192×10^3	4	2^4	9	2^5	12	2^4	34	6	2^8	1	2^{16}	0	4

is estimated using the Lattice Estimator⁴.

Failure Probability. FHE schemes introduce noise during encryption to ensure security. As homomorphic operations are applied, this noise accumulates, and decryption remains correct only while the noise stays below a defined threshold. We quantify the noise magnitude using the ℓ_2 norm. In practice, it is standard to bound the decryption failure probability below 2^{-32} to guarantee reliability [20]. A complete correctness proof of Tetris, together with the noise-tracking theorem and its proof, is provided in Appendix B.

Parameters. Under our recommended parameter sets Table 2, Tetris achieves 128-bit security, while reducing the algorithm decryption failure probability to below 2^{-40} . Alternative configurations can be selected to balance security, failure probability and efficiency.

5.2 Performance

We conduct a comprehensive experimental evaluation of Tetris against recent FHE LUT approaches. The evaluation covers: (1) PBS method in CCS'24 [6], (2) PBS-tree method in TCHES'25 [67], (3) the prior CMux-tree approach in CCS'25 [69] and our proposed Tetris framework, and (4) the recent CKKS-based LUT method in CRYPTO'25 [1].

⁴<https://github.com/malb/lattice-estimator>

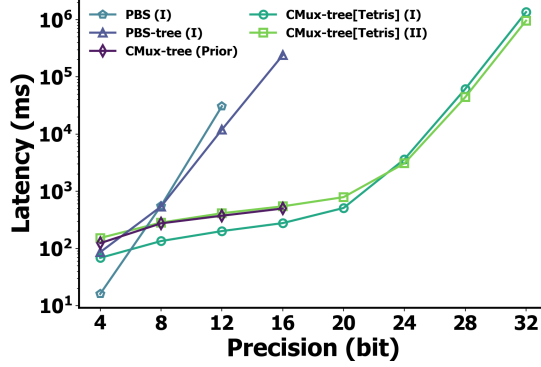


Figure 4: FHE LUT Runtime Results

Environment Setups. Most of our benchmarks can be executed on a desktop computer. However, due to the large memory requirements of comparative baselines—for example, the PBS-tree method [67] produces evaluation keys and table storage of approximately 40 GB—we conducted experiments on both a desktop computer and a server. The desktop computer is equipped with an Intel i7-13700 CPU @ 2.10 GHz and 24 GB memory, while the server comprises 112 Intel Xeon Gold 6258R CPUs @ 2.70 GHz with 1 TB memory.

For a fair comparison with prior works, all experiments are restricted to a single thread, and the reported results are averaged over more than 100 runs. The performance results are summarized in Table 3 and Figure 4.

Table 3: FHE LUT Runtime Results

Methods	Environment	4 bits	8 bits	12 bits	16 bits	20 bits	24 bits	28 bits	32 bits
PBS [6]	PC	9.56ms	275.50ms	—	—	—	—	—	—
	Server	16.08ms	557.91ms	30.71s*	—	—	—	—	—
PBS-tree [67]	PC	—	—	—	—	—	—	—	—
	Server	86.31ms	541.49ms	11.97s†	239.97s†	—	—	—	—
CMux-tree [69]	PC	68.88ms	137.85ms	206.61ms	277.48ms	—	—	—	—
	Server	123.96ms	274.92ms	373.88ms	499.91ms	—	—	—	—
CMux-tree [Tetris]	PC [Set I]	39.18ms	83.79ms	132.64ms	163.51ms	300.38ms	2.16s	—	—
	Server [Set I]	68.31ms	135.46ms	201.34ms	278.22ms	511.49ms	3.63s	61.51s	1360.81s
	Server-op‡ [Set II]	—	—	—	—	—	0.54s	6.40s	143.46s
CKKS LUT§ [1]	PC	—	—	—	—	—	—	—	—
	Server	46.70s	379.91s	—	—	—	—	—	—

* PBS method supports LUTs of up to 11 bits, and the reported runtime of 30.71s corresponds to this upper limit.

† The PBS-tree approach [67] reports performance results only for 4-bit and 8-bit LUTs, while in principle supporting up to 16-bit. The results for the 12-bit and 16-bit cases are estimated based on the required number of basic operations.

‡ Server-op integrates the high-precision optimizations presented in Section 5.3 under 8-thread execution.

§ The scheme theoretically claims support to 14-bit precision, but the implementation achieves a maximum of 8-bit.

4 Bits: PBS Is Enough. At low precision, PBS achieves the best performance. In this setting, the overhead of the more complex circuit bootstrapping in CMux-tree-based approaches (including Tetris) is not amortized by the LUT evaluation. Tetris records a latency of 39.18 ms, which is $4.1\times$ slower than PBS. However, it is already $1.76\times$ and $1.26\times$ faster than the prior CMux-tree implementation and the PBS-tree method, respectively. This initial gap is inherent

to the CBS paradigm, which pays a higher fixed cost in exchange for better asymptotic scaling.

8–16 Bits: Tetris Takes the Lead. As the precision increases to 8 bits (7 bits exactly, not shown in the table), Tetris establishes a clear and persistent advantage. The performance divergence becomes more pronounced, highlighting the fundamental limitations of competing methods:

- Against PBS, Tetris is $3.3\times$ – $152\times$ faster. PBS reaches its limit at 11 bits due to exponential parameter growth required for correctness.
- Against PBS-tree, Tetris is $4\times$ – $87\times$ faster. Although PBS-tree addresses parameter issue via decomposition, the exponential increase in PBS numbers makes it unscalable.
- Against the prior CMux-tree approach, Tetris achieves up to $2\times$ speedup (135.46 ms vs. 274.92 ms), highlighting the benefits of batched CBS and improved noise management.

The experiment results demonstrate that the CMux-tree paradigm—and in particular Tetris—soon becomes the only practical choice for medium-precision TFHE LUT. However, CKKS-based LUT method remains a notable outlier. While Tetris outperforms CKKS LUT by up to $2,901\times$ in latency at 8 bits, CKKS enjoys SIMD throughput by packing up to 65,536 messages per ciphertext. In other words, it remains a good choice for evaluating a *same* LUT on *large enough* volumes of messages.

Beyond 16 Bits: High-Precision Challenges. For precisions beyond 16 bits, Tetris is the only method that continues to function. However, the exponential complexity of the CMux-tree evaluation step ($O(2^\delta)$ for δ -bit precision) dominates runtime, making 32-bit LUTs inefficient without further optimization. As shown in Figure 5, the computational bottleneck shifts from circuit bootstrapping toward CMux evaluation. We therefore discuss optimization techniques for high-precision LUTs in Section 5.3 and pruning strategies for practical FHE instruction set architecture in Section 6.2.

5.3 Optimization for High-Precision

Although the CMux-tree method reduces the number of bootstrapping calls to linear, the evaluation step still involves an exponential number of CMux gates. As shown in Figure 5, for precisions below 16 bits, the latency of the evaluation step is negligible compared to CBS. Around 20-bit precision, however, the ratio between LUT evaluation and CBS grows to roughly 1:2, and beyond 24 bits, the evaluation step quickly dominates overall runtime with exponential growth. To enable efficient 32-bit LUT evaluation, we propose two complementary optimizations.

Minimizing Gadget Length in the Evaluation Step. Prior work [71, 69] empirically observed that conversion and evaluation steps contribute little to total latency, leading to parameter tuning strategies that minimized the gadget length ℓ of the refresh step. The cost of PBSmanyLUT is $n(k+1)(\ell+1)$ NTTs, so it is reasonable when evaluation latency is marginal.

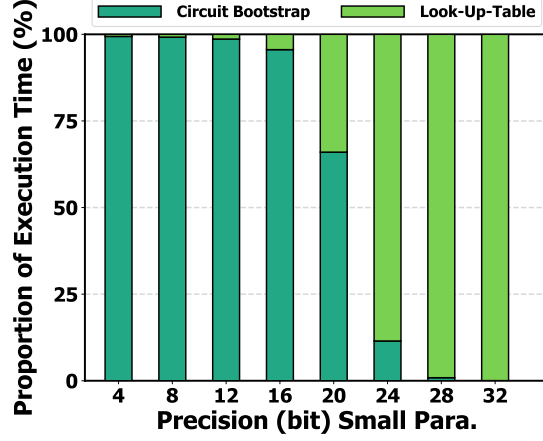


Figure 5: Proportion of CBS and LUT Execution Times

Our experiments confirm these observations but also suggest a different strategy for high precision (24 bits or more): shift the optimization focus to the evaluation step. Specifically, we increase the gadget length of the refresh step, sacrificing some efficiency there in exchange for reduced overall accumulated noise. This configuration allows us to minimize the decomposition length of the evaluation step to 1 (from Set I to Set II). As a result, under the same security and failure probability, we achieve a $1.5\times$ speedup, see Table 8.

Parallel Execution. In addition to parameter optimization, we explore the potential of parallel execution in CBS and CMux-tree evaluation. Unlike PBS-tree methods, where each PBS input depends on the packing of the previous outputs, the inputs to CBS refresh are independent. That give us an opportunity to parallel execute Tetris.

We propose a *thread-prioritized strategy*. Let τ_1 denote the batched degree of CBS and τ_2 the number of threads. For a δ -bit LUT, we first fully utilize τ_2 threads; each thread processes $\tau_1 = \min\left\{\lceil \frac{\delta}{\tau_2} \rceil, \max\{\tau_1\}\right\}$ bits per bootstrapping and executes $\lceil \frac{\delta}{\tau_1 \cdot \tau_2} \rceil$ bootstrappings. The CMux-tree is further partitioned so that evaluation also benefits from parallelism.

Finally, we evaluated our optimizations under 1-, 2-, 4-, and 8-thread configurations. As reported in Table 3, it achieves up to a $9.5\times$ speedup with 8 threads. A more comprehensive benchmarking of Tetris—covering precision levels (4–32 bits), parameterizations (Set I and Set II), and parallelism (1–8 threads)—is provided Appendix D (Table 8).

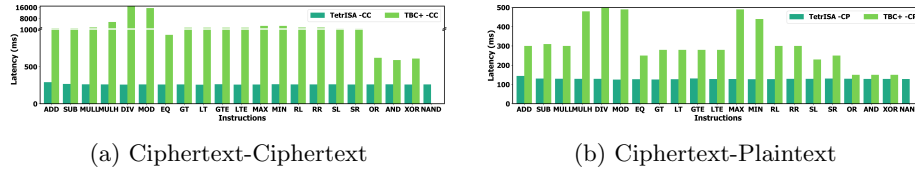


Figure 6: Performance Comparison of TetrISA and TBC+ FHE-ISA for 8-bit Bivariate / Univariate Instructions.

6 FHE Instruction Set Architecture

6.1 TetrISA

Evaluating instructions via FHE LUTs is straightforward. For an n -bit instruction, if only one operand is encrypted, it can be implemented by an n -to- n LUT; if both operands are encrypted, the corresponding LUT size grows to $2n$ -to- n (with a latency roughly half that of a full $2n$ -to- $2n$ LUT). Based on Tetris, we implemented the first circuit bootstrapping based FHE instruction set⁵, TetrISA. TetrISA extends the scope of recent FHE-ISA work [67, 69], supporting more than 20 instructions (Table 4), including arithmetic, logical, comparison, and bitwise operators. For detailed descriptions of these instructions, please refer to Table 7 in Appendix D.

Table 4: Supported Instructions in TetrISA

Type	Instructions
Arithmetic	ADD, SUB, MULL, MULH, DIV, MOD
Judging	EQ, GT, LT, GTE, LTE, MAX, MIN
Shift	RL, RR, SL, SR
Bitwise	OR, AND, XOR, NAND
Customized	SIGN, MOVE

Comparison with Prior FHE-ISA. Compared with the recent FHE-ISA by Trama et al. [67], TetrISA offers more flexible operand configurations. Specifically, it supports all three input combinations: ciphertext-ciphertext (CC), ciphertext-plaintext (CP), and plaintext-ciphertext (PC). In contrast, TBC+ [67] excluded the PC case. This omission limits the expressiveness for certain asymmetric operations, such as modular reduction, division, subtraction, and shifts. Some of them must instead be emulated using the CC mode at significant performance cost.

Figure 6 reports the performance of TetrISA under 8-bit bivariate and univariate settings, measured under the same precision limits as TBC+ [67].

⁵Wang et al. [69] proposed the concept of a circuit bootstrapping based FHE-ISA but only provided a theoretical estimation without implementation.

TetrISA achieves up to **65** $\times$ speedup, with the most significant improvements observed in complex operations such as division (65 $\times$) and modulus reduction (60 $\times$). The complete measured runtime results (including PC mode) are provided in Table 9 in Appendix D.

Comparison with Other FHE Compilers. We further compared TetrISA with representative open-source FHE compilers: Juliet [38], Cingulata [12], E3 [19] and Concrete [74]. Juliet and Cingulata translate programs directly into Boolean FHE operators. E3 adopts a hybrid approach with both arithmetic and Boolean operators. Concrete supports Boolean operators and low-precision LUTs via PBS.

For a fair comparison, we selected standard benchmark tasks used across prior works: *Max.* (find the maximum element of an array), *SquareS.* (sum of squares), *Avg.* (mean of all elements), and *Bub.Sort* (swap adjacent out-of-order elements until sorted). The test program takes an array of length five as input. As shown in Table 5, TetrISA achieves the best efficiency across all four benchmarks, outperforming prior schemes by a factor of **3–40** $\times$.

Remark 2. *Several competitors run only on memory-rich machines, so the results are summarized under the server environment. In addition, TetrISA allows execution on desktop computers, further surpassing the server baseline. For completeness, both PC and server results of TetrISA are reported in Table 5.*

Table 5: Programs Execution Time

Scheme	Max.	SquareS.	Avg.	Bub.Sort
Juliet [38]	18.10/13.7 $\times$	56.80/21.5 $\times$	–	68.70/12.9 $\times$
Cingulata [12]	4.28/3.2 $\times$	19.00/7.2 $\times$	24.33/16.7 $\times$	16.92/3.2 $\times$
E3 [19]	7.16/5.4 $\times$	19.93/7.6 $\times$	8.43/5.8 $\times$	34.80/6.5 $\times$
Concrete [74]	49.36/37.4 $\times$	17.41/6.6 $\times$	–	213.79/40.2 $\times$
TBC+ [67]	10.86/8.2 $\times$	12.80/4.9 $\times$	5.58/3.8 $\times$	41.66/7.8 $\times$
TetrISA	1.32/1.0$\times$	2.64/1.0$\times$	1.46/1.0$\times$	5.19/1.0$\times$
TetrISA_{PC}	0.83	1.65	0.91	3.20

(a) Program running time (in seconds) and speedup analysis comparing TetrISA, TBC+ FHE-ISA, and related FHE compilers across benchmark workloads.

6.2 High-Precision Instructions and Pruning

With TetrIS, TetrISA scales to 32-bit instructions. However, a n -bit bivariate operation corresponds to a LUT of size 2^{2n} . This poses two challenges: (a) improving the efficiency of supported but costly instructions (16-bit bivariate / 32-bit univariate), and (b) enabling 32-bit bivariate operations, which are unfeasible with naive 64-bit LUT evaluation.

To address these issues, we introduce *LUT pruning*, which eliminates redundant branches in the evaluation tree.

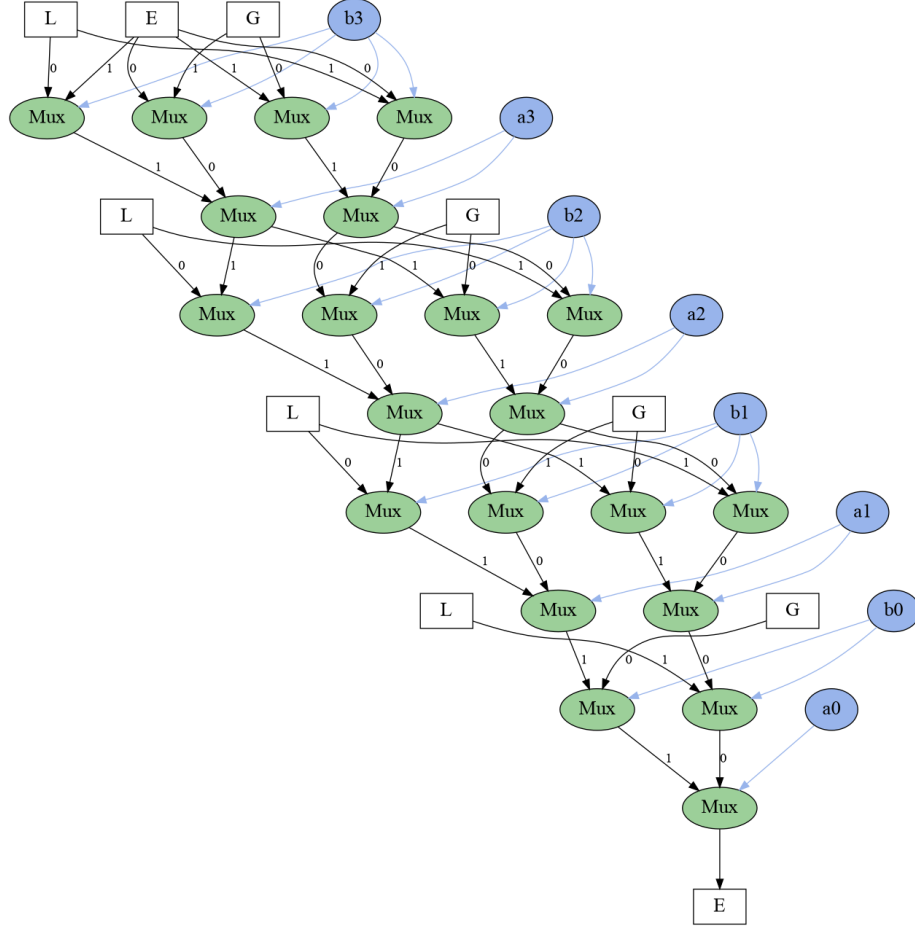


Figure 7: General structure of the pruned comparator LUT for comparison instructions (GT, GTE, LT, LTE, EQ).

As a proof of concept, we apply pruning to all comparison instructions (GT, GTE, LT, LTE, EQ). The general structure of the pruned comparator is shown in Figure 7.

We begin by initializing three states— G (greater), E (equal), and L (less)—according to the comparison type. The two inputs are then processed from the least to the most significant bits in 4-bit chunks ($a_0a_1a_2a_3, b_0b_1b_2b_3$). Each chunk serves as selector control bits for one iteration: if the inputs are plaintexts, the selector is realized by a plaintext gate; if ciphertext, by a CMux gate. The output E from each round is carried forward as the E input to the next round. After all iterations, extracting the final LWE ciphertext from E yields the comparison result.

Through this pruning strategy, TetrISA not only accelerates existing 16-bit

Table 6: Performance of Optimized GT

Width	Mode	TetrISA-op	TetrISA
16	GTE _{CC}	1.14/1.0×	482.39/423.1×
	GTE _{CC}	2.27	—
32	GTE _{CP}	1.16/1.0×	975.45/840.9×
	GTE _{PC}	1.17/1.0×	975.45/833.7×

(a) Performance comparison between the optimized and trivial implementations (in seconds). “Width” denotes the bit width of the data, while “Mode” indicates whether the two inputs are ciphertexts or plaintexts.

and 32-bit comparison instructions, but also makes 32-bit bivariate comparisons tractable. Since all comparison operators (GT, GTE, LT, LTE, EQ) exhibit nearly identical runtimes, we report results only for GTE in Table 6. The results demonstrate that pruning achieves up to a **915**× speedup, enabling a practical execution of 32-bit bivariate comparisons within 3 seconds. Extending pruning to other instructions is left as our future work.

6.3 User-Defined Operations

Some computational functions, while not strictly necessary for instruction set completeness (as they can be composed from other primitives), are frequently invoked in practice. Providing specialized implementations for such functions can therefore improve overall performance in typical workloads. TetrISA includes several such user-defined instructions, including Sign and MOVE.

As an example, we consider the Sign function. Since our data type is based on unsigned integers, we slightly adapt the conventional definition. Given a plaintext threshold p , an encrypted input x is mapped as follows:

$$\text{Sign}(x, p) = \begin{cases} -1, & x < p, \\ 0, & x = p, \\ 1, & x > p. \end{cases}$$

As a result, TetrISA evaluates the 32-bit Sign function in 528 ms. In addition, MOVE instruction completes in 1.77 μ s.

Comparison. To the best of our knowledge, the only competing scheme claiming support for a 32-bit Sign function is CKKS LUT [1], but no implementation is provided. For fairness comparison, we normalize its reported runtime using the ratio between claimed and measured LUT performance, obtaining a baseline of 1538.2 s. In contrast, TetrISA evaluates 32-bit Sign instruction in 527.54 ms, representing a **2916**× speedup. While CKKS can pack up to 65,536 messages and thus achieves higher throughput, TetrISA delivers a practical low latency, with throughput only 22.5× lower even in the worst case.

7 Conclusion

Fully Homomorphic Encryption (FHE) has long been viewed as a cornerstone for secure computation. However, its deployment continues to be hindered by the dual challenges of efficiency and programmability. In this work, we introduced *Tetris*, a versatile and efficient TFHE LUT framework, and *TetrISA*, the first high-precision FHE Instruction Set Architecture built upon it. Our results demonstrate that expressive and high-performance FHE instruction sets are no longer a distant aspiration but a concrete step toward practical FHE processor.

At the same time, significant challenges remain. Scaling LUT-based methods to 64-bit precision is still an open question. Existing methods suffer from exponential blowup—even CBS-based approaches defer this explosion—leading to prohibitive costs on the order of $\sim 200,000$ years of computation and hundreds of exabytes of storage. Breaking this precision limit will likely require fundamentally new paradigms for FHE evaluation, possibly through novel encoding schemes, alternative bootstrapping strategies, or hybrid designs that transcend current asymptotic limitations.

References

- [1] Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General functional bootstrapping using ckks. In *Annual International Cryptology Conference*, pages 304–337. Springer, 2025.
- [2] Jacob Alperin-Sheriff and Chris Peikert. Faster bootstrapping with polynomial error. In *Annual Cryptology Conference*, pages 297–314. Springer, 2014.
- [3] Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. PIR with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and Privacy*, pages 962–979, San Francisco, CA, USA, May 21–23, 2018. IEEE Computer Society Press.
- [4] David W Archer, José Manuel Calderón Trilla, Jason Dagit, Alex Malozemoff, Yuriy Polyakov, Kurt Rohloff, and Gerard Ryan. Ramparts: A programmer-friendly system for building homomorphic encryption applications. In *Proceedings of the 7th acm workshop on encrypted computing & applied homomorphic cryptography*, pages 57–68, 2019.
- [5] Loris Bergerat, Anas Boudi, Quentin Bourgerie, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Parameter Optimization and Larger Precision for (T)FHE. *Journal of Cryptology*, 36:28, 2023.
- [6] Loris Bergerat, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, Adeline Roux-Langlois, and Samuel Tap. New secret keys for enhanced performance in (t)fhe. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS ’24*, page 2547–2561, New York, NY, USA, 2024. Association for Computing Machinery.
- [7] Song Bian, Zian Zhao, Ruiyu Shen, Zhou Zhang, Ran Mao, Dawei Li, Yizhong Liu, Masaki Waga, Kohei Suenaga, Zhenyu Guan, et al. Chloe: Loop transformation over fully homomorphic encryption via multi-level vectorization and control-path reduction. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 2395–2413. IEEE, 2025.
- [8] Song Bian, Zian Zhao, Zhou Zhang, Ran Mao, Kohei Suenaga, Yier Jin, Zhenyu Guan, and Jianwei Liu. Heir: A unified representation for cross-scheme compilation of fully homomorphic computation. *Cryptology ePrint Archive*, 2023.
- [9] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. Simulating Homomorphic Evaluation of Deep Learning Predictions. In Shlomi Dolev, Danny Hendler, Sachin Lodha, and Moti Yung, editors, *Cyber Security Cryptography and Machine Learning*, volume 11527, pages 212–230. Springer, 2019.

- [10] Zvika Brakerski. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417, pages 868–886. Springer, 2012.
- [11] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) Fully Homomorphic Encryption without Bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, page 309–325. ACM, 2012.
- [12] Sergiu Carpov, Paul Dubrulle, and Renaud Sirdey. Armadillo: a compilation chain for privacy preserving applications. In *Proceedings of the 3rd International Workshop on Security in Cloud Computing*, pages 13–19, 2015.
- [13] Sergiu Carpov, Malika Izabachène, and Victor Mollimard. New techniques for multi-value input homomorphic evaluation and applications. In *Cryptographers’ Track at the RSA Conference*, pages 106–126. Springer, 2019.
- [14] Hao Chen, Ilaria Chillotti, and Ling Ren. Onion Ring ORAM: Efficient Constant Bandwidth Oblivious RAM from (Leveled) TFHE. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’19, page 345–360. ACM, 2019.
- [15] Hao Chen, Zhicong Huang, Kim Laine, and Peter Rindal. Labeled PSI from fully homomorphic encryption with malicious security. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 1223–1237, Toronto, ON, Canada, October 15–19, 2018. ACM Press.
- [16] Hao Chen, Kim Laine, and Peter Rindal. Fast private set intersection from homomorphic encryption. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1243–1255, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
- [17] J. H. Cheon, D. Kim, Y. Kim, and Y. Song. Ensemble Method for Privacy-Preserving Logistic Regression Based on Homomorphic Encryption. *IEEE Access*, 6:46938–46948, 2018.
- [18] Seonyoung Cheon, Yongwoo Lee, Dongkwan Kim, Ju Min Lee, Sunchul Jung, Taekyung Kim, Dongyoon Lee, and Hanjun Kim. {DaCapo}: Automatic bootstrapping management for efficient fully homomorphic encryption. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 6993–7010, 2024.
- [19] Eduardo Chielle, Oleg Mazonka, Homer Gamil, Nektarios Georgios Tsoutsos, and Michail Maniatakos. E3: A framework for compiling c++ programs with encrypted operands. *Cryptology ePrint Archive*, 2018.

- [20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster Fully Homomorphic Encryption: Bootstrapping in Less Than 0.1 Seconds. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology – ASIACRYPT 2016*, volume 10031, pages 3–33. Springer, 2016.
- [21] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster Packed Homomorphic Operations and Efficient Circuit Bootstrapping for TFHE. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 377–408, Cham, 2017. Springer International Publishing.
- [22] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *Journal of Cryptology*, 33:34–91, 2020.
- [23] Ilaria Chillotti, Marc Joye, and Pascal Paillier. Programmable Bootstrapping Enables Efficient Homomorphic Inference of Deep Neural Networks. In Shlomi Dolev, Oded Margalit, Benny Pinkas, and Alexander Schwarzmann, editors, *Cyber Security Cryptography and Machine Learning*, pages 1–19, Cham, 2021. Springer International Publishing.
- [24] Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Improved Programmable Bootstrapping with Larger Precision and Efficient Arithmetic Circuits for TFHE. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021*, pages 670–699. Springer, 2021.
- [25] Kelong Cong, Radames Cruz Moreno, Mariana Botelho da Gama, Wei Dai, Ilia Iliashenko, Kim Laine, and Michael Rosenberg. Labeled PSI from homomorphic encryption with reduced computation and communication. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 1135–1150, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.
- [26] Meghan Cowan, Deeksha Dangwal, Armin Alaghi, Caroline Trippel, Vincent T Lee, and Brandon Reagen. Porcupine: A synthesizing compiler for vectorized homomorphic encryption. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 375–389, 2021.
- [27] Eric Crockett, Chris Peikert, and Chad Sharp. Alchemy: A language and compiler for homomorphic encryption made easy. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1020–1037, 2018.
- [28] Ashwini Dalvi, Apoorva Jain, Smit Moradiya, Riddhisha Nirmal, Jay Sanghavi, and Irfan Siddavatam. Securing neural networks using homomorphic encryption. In *2021 International Conference on Intelligent Technologies (CONIT)*, pages 1–7, 2021.

- [29] Thomas de Ruijter, Jan-Pieter D’Anvers, and Ingrid Verbauwhede. Don’t be mean: Reducing approximation noise in tfhe through mean compensation. *Cryptology ePrint Archive*, 2025.
- [30] Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015*, volume 9056, pages 617–640. Springer, 2015.
- [31] Junfeng Fan and Frederik Vercauteren. Somewhat Practical Fully Homomorphic Encryption. IACR Cryptology ePrint Archive, Report 2012/144, 2012. <https://eprint.iacr.org/2012/144>.
- [32] Ben Fisch, Arthur Lazzaretti, Zeyu Liu, and Charalampos Papamanthou. Thorpir: Single server pir via homomorphic thorp shuffles. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS ’24*, page 1448–1462, New York, NY, USA, 2024. Association for Computing Machinery.
- [33] Craig Gentry. Fully Homomorphic Encryption Using Ideal Lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, page 169–178. ACM, 2009.
- [34] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013*, volume 8042, pages 75–92. Springer, 2013.
- [35] Google. Fully Homomorphic Encryption (FHE) C++ Transpiler. <https://github.com/google/fully-homomorphic-encryption/tree/main/transpiler>.
- [36] Google. HEIR: Homomorphic Encryption Intermediate Representation. <https://github.com/google/heir>.
- [37] Shruthi Gorantala, Rob Springer, Sean Purser-Haskell, William Lam, Royce Wilson, Asra Ali, Eric P Astor, Itai Zukerman, Sam Ruth, Christoph Dibak, et al. A general purpose transpiler for fully homomorphic encryption. *arXiv preprint arXiv:2106.07893*, 2021.
- [38] Charles Gouert, Dimitris Mouris, and Nektarios Georgios Tsoutsos. Juliet: A configurable processor for computing on encrypted data. *IEEE Transactions on Computers*, 73(9):2335–2349, 2024.
- [39] Charles Gouert and Nektarios Georgios Tsoutsos. Romeo: Conversion and evaluation of hdl designs in the encrypted domain. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2020.

- [40] Zhenyu Guan, Ran Mao, Qianyun Zhang, Zhou Zhang, Zian Zhao, and Song Bian. Autohog: Automating homomorphic gate design for large-scale logic circuit evaluation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(7):1971–1983, 2024.
- [41] Antonio Guimarães, Edson Borin, and Diego F Aranha. Revisiting the functional bootstrap in tffe. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 229–253, 2021.
- [42] Antonio Guimarães, Edson Borin, and Diego F. Aranha. MOSFHET: Optimized Software for FHE over the Torus. *Journal of Cryptographic Engineering*, 14(3):577–593, July 2024.
- [43] Jae Hyung Ju, Jaiyoung Park, Jongmin Kim, Minsik Kang, Donghwan Kim, Jung Hee Cheon, and Jung Ho Ahn. Neujeans: Private neural network inference with joint optimization of convolution and the bootstrapping. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4361–4375, 2024.
- [44] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. {GAZELLE}: A low latency framework for secure neural network inference. In *27th USENIX security symposium (USENIX security 18)*, pages 1651–1669, 2018.
- [45] Joon-Woo Lee, Hyungchul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, and Jong-Seon No. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access*, 10:30039–30054, 2022.
- [46] Joon-Woo Lee, Eunsang Lee, Young-Sik Kim, and Jong-Seon No. Rotation key reduction for client-server systems of deep neural network on fully homomorphic encryption. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 36–68, Singapore, 2023. Springer Nature Singapore.
- [47] Kang Hoon Lee and Ji Won Yoon. Homomorphic field trace revisited: Breaking the cubic noise barrier. *Cryptology ePrint Archive*, 2025.
- [48] Keewoo Lee and Yongdong Yeo. SophOMR: Improved oblivious message retrieval from SIMD-aware homomorphic compression. *Cryptology ePrint Archive*, Paper 2024/1814, 2024.
- [49] Yongwoo Lee, Seonyoung Cheon, Dongkwan Kim, Dongyoon Lee, and Hanjun Kim. {ELASM}:: {Error-Latency-Aware} scale management for fully homomorphic encryption. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4697–4714, 2023.

- [50] Yongwoo Lee, Seonyeong Heo, Seonyoung Cheon, Shinnung Jeong, Changsu Kim, Eunkyung Kim, Dongyoon Lee, and Hanjun Kim. Hecate: performance-aware scale optimization for homomorphic encryption compiler. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 193–204. IEEE, 2022.
- [51] Jian Liu, Jingyu Li, Di Wu, and Kui Ren. Pirana: Faster multi-query pir via constant-weight codes. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4315–4330. IEEE, 2024.
- [52] Z Liu and Y Wang. Relaxed functional bootstrapping: a new perspective on bgv. Technical report, BFV bootstrapping. Cryptology ePrint Archive, Paper 2024/172, 2024.
- [53] Zeyu Liu and Eran Tromer. Oblivious message retrieval. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 753–783, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Heidelberg, Germany.
- [54] Zeyu Liu, Eran Tromer, and Yunhao Wang. Group oblivious message retrieval. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4367–4385, 2024.
- [55] Zeyu Liu and Yunhao Wang. Amortized Functional Bootstrapping in Less than 7 ms, with $\tilde{O}(1)$ Polynomial Multiplications. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 101–132, Singapore, 2023. Springer Nature Singapore.
- [56] Wen-jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. Pegasus: bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1057–1073. IEEE, 2021.
- [57] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 1–23. Springer, 2010.
- [58] Kotaro Matsuoka. TFHEpp: pure C++ implementation of TFHE cryptosystem. <https://github.com/virtualsecureplatform/TFHEpp>, 2020.
- [59] Kotaro Matsuoka, Ryotaro Banno, Naoki Matsumoto, Takashi Sato, and Song Bian. Virtual secure platform: A {Five-Stage} pipeline processor over {TFHE}. In *30th USENIX security symposium (USENIX Security 21)*, pages 4007–4024, 2021.
- [60] T Meeradevi, K Mohanraj, BM Mourish, V Santhosh Siva, Ravi Samikannu, and S Sasikala. Design of 32 bit risc v processor. In *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–7. IEEE, 2024.

- [61] Samir Jordan Menon and David J. Wu. SPIRAL: Fast, high-rate single-server PIR via FHE composition. In *2022 IEEE Symposium on Security and Privacy*, pages 930–947, San Francisco, CA, USA, May 22–26, 2022. IEEE Computer Society Press.
- [62] Microsoft. EVA - Compiler for Microsoft SEAL. <https://github.com/microsoft/EVA>.
- [63] Michael S Paterson and Larry J Stockmeyer. On the number of nonscalar multiplications necessary to evaluate polynomials. *SIAM Journal on Computing*, 2(1):60–66, 1973.
- [64] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.
- [65] Ronald L Rivest, Len Adleman, Michael L Dertouzos, et al. On data banks and privacy homomorphisms. *Foundations of secure computation*, 4(11):169–180, 1978.
- [66] DV Soundari, MK Shanker Ganesh, Indira Raman, and R Karthick. Enhancing network-on-chip performance by 32-bit risc processor based on power and area efficiency. *Materials Today: Proceedings*, 45:2713–2720, 2021.
- [67] Daphné Trama, Aymen Boudguiga, Pierre-Emmanuel Clet, Renaud Sirdey, and Nicolas Ye. Designing a general-purpose 8-bit (t) fhe processor abstraction. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):535–578, 2025.
- [68] Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi. {HECO}: Fully homomorphic encryption compiler. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4715–4732, 2023.
- [69] Ruida Wang, Jincheol Ha, Xuan Shen, Xianhui Lu, Chunling Chen, Kunpeng Wang, and Jooyoung Lee. Refined tfhe leveled homomorphic evaluation and its application. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS '25*. ACM, 2025.
- [70] Ruida Wang, Benqiang Wei, Zhihao Li, Xianhui Lu, and Kunpeng Wang. Tfhe bootstrapping: Faster, smaller and time-space trade-offs. In *Australasian Conference on Information Security and Privacy*, pages 196–216. Springer, 2024.
- [71] Ruida Wang, Yundi Wen, Zhihao Li, Xianhui Lu, Benqiang Wei, Kun Liu, and Kunpeng Wang. Circuit Bootstrapping: Faster and Smaller. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 342–372, Cham, 2024. Springer Nature Switzerland.

- [72] Benqiang Wei, Xianhui Lu, Ruida Wang, Kun Liu, Zhihao Li, and Kunpeng Wang. Thunderbird: Efficient homomorphic evaluation of symmetric ciphers in 3gpp by combining two modes of tfhe. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(3):530–573, 2024.
- [73] Xue Yang, Ruida Wang, Depan Peng, Kun Liu, Xianhui Lu, and Xiaohu Tang. Pircor: Communication-optimal hintless single-server pir via homomorphic rotation. *Cryptology ePrint Archive*, 2025.
- [74] Zama. Concrete: TFHE Compiler that converts python programs into FHE equivalent, 2022. <https://github.com/zama-ai/concrete>.

A Algorithms

A.1 Key Switching

Let $\mathbf{S}$ and $\mathbf{S}'$ be two GLWE secret keys of dimensions k and k' , respectively, and of the same polynomial size N . The GLWE keyswitching from $\mathbf{S}$ to $\mathbf{S}'$ changes a GLWE ciphertext of M under $\mathbf{S}$ to another GLWE ciphertext of M under $\mathbf{S}'$ using the GLWE keyswitching key $\{\text{GLew}_{\mathbf{S}'}(S_i)\}_{i=1}^k$, a set of k GLWE ciphertexts of S_i , $i = 1, \dots, k$. The precise description of the algorithm is given in Algorithm 7.

Algorithm 7: GLWE keyswitching GLWE_KS

Input: $\mathbf{C} = \text{GLWE}_{\mathbf{S}}(M)$ under $\mathbf{S} = (S_1, \dots, S_k)$
Input: $\text{KS}_{\mathbf{S} \rightarrow \mathbf{S}'}[i] = \text{GLew}_{\mathbf{S}'}^{(B, \ell)}(S_i)$ for $i = 1, \dots, k$ with decomposition base B and length ℓ under $\mathbf{S}' = (S'_1, \dots, S'_{k'})$
Output: $\mathbf{C}' = \text{GLWE}_{\mathbf{S}'}(M)$

- 1 $\mathbf{C} = (A_1, \dots, A_k, A_{k+1})$
- 2 $\text{KS}_{\mathbf{S} \rightarrow \mathbf{S}'}[i][j] = \text{GLWE}_{\mathbf{S}'}\left(\frac{q}{B^j} \cdot S_i\right)$ for $i \in [k]$ and $j \in [\ell]$
- 3 $\mathbf{C}' \leftarrow (0, \dots, 0, A_{k+1}) = \text{GLWE}_{\mathbf{S}'}^0(A_{k+1}) \in \mathcal{R}_{q,N}^{k'+1}$
- 4 **for** $i = 1$ **to** k **do**
- 5 Decompose A_i as $\sum_{j=1}^{\ell} A'_{i,j} \cdot \frac{q}{B^j} + E'_i$ with $\|A'_{i,j}\|_{\infty} \leq \frac{B}{2}$ and $\|E'_i\|_{\infty} \leq \frac{q}{2B^{\ell}}$
- 6 $\mathbf{C}' \leftarrow \mathbf{C}' - \sum_{j=1}^{\ell} A'_{i,j} \cdot \text{KS}_{\mathbf{S} \rightarrow \mathbf{S}'}[i][j]$
- 7 **return** $\mathbf{C}'$

A.2 Automorphism

The automorphism and trace can be defined on the polynomial ring $\mathcal{R}_N$ and its residue ring $\mathcal{R}_{q,N}$. For $d \in \mathbb{Z}_{2N}^{\times}$, the automorphism τ_d on $\mathcal{R}_N$ (or $\mathcal{R}_{q,N}$) is defined by $\tau_d : \mu(X) \mapsto \mu(X^d)$, and the trace function Tr on $\mathcal{R}_N$ (or $\mathcal{R}_{q,N}$) is defined by

$$\text{Tr}(\mu(X)) := \sum_{d \in \mathbb{Z}_{2N}^{\times}} \tau_d(\mu(X)) = N\mu_0. \quad (1)$$

The automorphism can be homomorphically evaluated by GLWE keyswitching as described in Algorithm 8.

A.3 Programmable Bootstrapping

The programmable bootstrapping (PBS) of TFHE supports an extra functionality that evaluates a function for free during the bootstrapping. Suppose that an LWE ciphertext $\mathbf{c} = (a_1, \dots, a_n, b) \in \mathbb{Z}_q^{n+1}$ of a phase $\mu = \Delta m + e$ under a secret key $\mathbf{s} = (s_1, \dots, s_n) \in \mathbb{B}^n$ is given. The PBS operation outputs a refreshed

Algorithm 8: Evaluating Automorphism EvalAuto($\mathbf{C}, d$)

Input: $\mathbf{C} = \text{GLWE}_{\mathbf{S}(X)}(M(X))$, $d \in \mathbb{Z}_{2N}^\times$
Input: $\text{AutoKey}_d = \text{KS}_{\mathbf{S}(X^d) \rightarrow \mathbf{S}(X)}$ with decomposition base B and length ℓ
Output: $\mathbf{C}' = \text{GLWE}_{\mathbf{S}(X)}(M(X^d))$
 $\mathbf{1} \ \mathbf{C} = (A_1, \dots, A_k, B)$
 $\mathbf{2} \ \mathbf{C}' \leftarrow (A'_1, \dots, A'_k, B') = (A_1(X^d), \dots, A_k(X^d), B(X^d))$
 $\mathbf{3} \ \mathbf{C}' \leftarrow \text{GLWE.KS}(\mathbf{C}', \text{AutoKey}_d)$
 $\mathbf{4} \ \text{return } \mathbf{C}'$

LWE ciphertext $\mathbf{c}' \in \mathbb{Z}_q^{kN}$ of the message $f(m)$ under a secret key $\mathbf{s}' \in \mathbb{B}^{kN}$ by the following steps.

1. Encode the function f on a new GLWE ciphertext under a different secret key $\mathbf{S}' \in \mathbb{B}_N[X]^k$. The half of the function values of f are redundantly encoded in the coefficients of the plaintext of the (trivial) GLWE ciphertext.
2. (Modulus switching) Compute $\tilde{\mathbf{c}} = (\tilde{a}_1, \dots, \tilde{a}_n, \tilde{b}) \in \mathbb{Z}_{2N}^{n+1}$ where

$$\tilde{a}_i = \lfloor a_i \cdot (2N)/q \rfloor \text{ and } \tilde{b} = \lfloor b \cdot (2N)/q \rfloor,$$

obtaining an LWE ciphertext of a phase $\tilde{\mu} \approx \lfloor \mu \cdot (2N)/q \rfloor$.

3. (Blind rotation) Multiply $X^{-\tilde{b} + \sum_{i=1}^n \tilde{a}_i s_i} = X^{-\tilde{\mu}}$ to the GLWE ciphertext encoding the function using a bootstrapping key $\{\text{GGSW}_{\mathbf{S}'}(s_i)\}_{i=1}^n$; multiply either 1 or $X^{-\tilde{a}_i}$ according to $s_i \in \{0, 1\}$ by the CMux gate.
4. (Sample extraction) Extract the constant term of the GLWE ciphertext, obtaining an LWE ciphertext of $f(m)$ under the secret key $\mathbf{s}' \in \mathbb{B}^{kN}$ which is a reordering of the coefficients of $\mathbf{S}'$.

Since $X^N = -1$ in the ring $\mathcal{R}_{q,N}$, it is only possible to evaluate a negacyclic function $f : \mathbb{Z}_p \rightarrow \mathbb{Z}_q$ such that $f(x + p/2) = -f(x)$ by encoding only half of the function values. To evaluate an arbitrary function, TFHE requires one padding bit of zero in the MSB of μ to guarantee $\tilde{\mu} < N$.

LWE Keyswitching. The input and output LWE dimensions might be different for the PBS operation. To improve the performance of PBS, it is common to use a smaller input LWE dimension than the output LWE dimension. Hence, one needs to switch the LWE dimension before the PBS operation, and this step is called the *LWE keyswitching*. In practice, the LWE keyswitching operation is performed only once just before the PBS operation to match the LWE dimension rather than after every PBS operation.

GLWE Keyswitching and Functional Keyswitching. It is possible to switch the key of GLWE ciphertexts. Such keyswitching operation is called

GLWE keyswitching. The core concept of GLWE keyswitching is the same with that of LWE keyswitching, but its polynomial operation can be accelerated by NTT/FFT. The detailed algorithm is given in Algorithm 7.

A.4 PBSmanyLUT and MV-PBS

PBS supports two single-input, multi-output techniques: (1) *MV-PBS*, which extracts a common factor f_0 from a target function set $\{f_i\}_{i=1}^k$ and computes all outputs by multiplying $\frac{f_i}{f_0}$ after bootstrapping, see Algorithm 9; (2) *PBSmanyLUT*, which clears ϑ noise positions in the bootstrapped ciphertext, enabling encoding of up to 2^ϑ functions in one PBS. The algorithm is given in Algorithm 10.

Algorithm 9: MV-PBS

Input: $\mathbf{c}_{\text{in}} = \text{LWE}_{\mathbf{s}}(m \cdot \Delta_{\text{in}}) = (a_1, \dots, a_n, a_{n+1} = b) \in \mathbb{Z}_q^{n+1}$
Input: $\text{BSK} = (\text{GGSW}_{\mathbf{s}'}(s_i))_{1 \leq i \leq n}$ under $\mathbf{S}' = (S'_1, \dots, S'_k)$
Input: $t + 1$ polynomials P_0 and P'_j for $j = 1, \dots, t$ such that $P'_j \cdot P_0 = P_j$ where P_j is the polynomial encoding f_j
Output: $\mathbf{c}_1, \dots, \mathbf{c}_t$ where $\mathbf{c}_j = \text{LWE}_{\mathbf{s}'}(f_j(m) \cdot \Delta_{\text{out}})$ and $\mathbf{s}'$ is the LWE secret key corresponding to $\mathbf{S}'$

```

1 for  $i = 1$  to  $n + 1$  do
2    $\tilde{a}_i = \lfloor a_i \cdot (2N)/q \rfloor$  /* ModSwitch */
3 Let  $\text{ACC} \leftarrow \text{GLWE}_{\mathbf{S}'}^0(P_0)$ 
4  $\text{ACC} \leftarrow \text{BlindRotate}(\text{ACC}, \{\tilde{a}_i\}_{i=1}^{n+1}, \text{BSK})$ 
5 for  $j = 1$  to  $t$  do
6    $\text{ACC}_j \leftarrow P'_j \cdot \text{ACC}$ 
7    $\mathbf{c}_j = \text{SampleExtract}(\text{ACC}_j)$ 
8 return  $\mathbf{c}_1, \dots, \mathbf{c}_t$ 

```

A.5 Circuit Bootstrapping

The circuit bootstrapping (CBS) refreshes and converts an LWE ciphertext of a single bit into the corresponding GGSW ciphertext [22]. After obtaining refreshed GGSW ciphertexts by CBS, the LHE mode can evaluate circuits efficiently by CMux gates. In this paper, we describe CBS in two steps: the Refr. and Conv. steps.

The Refr. step refreshes the input LWE ciphertext by PBS and outputs the corresponding Lev ciphertext. Given an LWE ciphertext $\text{LWE}_{\mathbf{s}}(\Delta m)$ of a single bit message m with some scaling factor Δ , one can compute $\text{Lev}_{\mathbf{s}}^{(B, \ell)}(m)$ by gathering its internal LWE ciphertexts $\text{LWE}_{\mathbf{s}}(v_j \cdot m)$ where $v_j = \lceil q/B^j \rceil$ for $j = 1, \dots, \ell$ using PBS. Since it computes ℓ PBS operations on the same LWE input with several output scaling factors, PBSmanyLUT can improve this step without increasing the PBS error.

Algorithm 10: PBSmanyLUT

Input: $\mathbf{c}_{\text{in}} = \text{LWE}_{\mathbf{s}}(m \cdot \Delta_{\text{in}}) = (a_1, \dots, a_n, a_{n+1} = b) \in \mathbb{Z}_q^{n+1}$ under $\mathbf{s} = (s_1, \dots, s_n)$ where $(\beta, m') \leftarrow \text{PTModSwitch}_q(m, \Delta_{\text{in}}, \vartheta)$
Input: $\text{BSK} = (\text{GGSW}_{\mathbf{S}'}(s_i))_{1 \leq i \leq n}$ under $\mathbf{S}' = (S'_1, \dots, S'_k)$ with decomposition base B_{pbs} and level ℓ_{pbs}
Input: $P_{(f_1, \dots, f_{2^\vartheta})}$: a redundant LUT for $f_1, \dots, f_{2^\vartheta}$
Output: $\mathbf{c}_1, \dots, \mathbf{c}_{2^\vartheta}$ where $\mathbf{c}_j = \text{LWE}_{\mathbf{s}'}((-1)^\beta \cdot f_j(m') \cdot \Delta_{\text{out}})$
1 and $\mathbf{s}'$ is the LWE secret key corresponding to $\mathbf{S}'$ **for** $i = 1$ **to** $n + 1$ **do**
2 $a'_i \leftarrow \left\lfloor \left\lceil \frac{a_i \cdot 2N \cdot 2^{-\vartheta}}{q} \right\rceil \cdot 2^\vartheta \right\rfloor_{2N}$ /* Using MC-ModSwitch */
3 $\mathbf{C}_{(f_1, \dots, f_{2^\vartheta})} \leftarrow \text{GLWE}_{\mathbf{S}'}^0(P_{(f_1, \dots, f_{2^\vartheta})})$
4 $\mathbf{C} \leftarrow \text{BlindRotate}(\mathbf{C}_{(f_1, \dots, f_{2^\vartheta})}, \{a'_i\}_{i=1}^{n+1}, \text{BSK})$
5 **for** $j = 1$ **to** 2^ϑ **do**
6 $\mathbf{c}_j \leftarrow \text{SampleExtract}_{j-1}(\mathbf{C})$

Next is the Conv. step converting the Lev ciphertext into the GGSW ciphertext. Chillotti et al. [21] used the private functional keyswitching to convert $\text{LWE}(v_j \cdot m)$ to $\text{GLWE}(v_j \cdot (-S_i \cdot m))$ when proposing the CBS algorithm, requiring $(k+1)\ell$ private functional keyswitching operations for each CBS operation.⁶

Since then, there have been many approaches to optimize the Conv. step. The first one is to precompute the multiplication result of the evaluation key to improve the speed of the private keyswitching itself at the cost of increased key size. For example, the TFHEpp library [58] adopts this technique. Another one is using public functional keyswitching combined with scheme switching to replace private functional keyswitching [14, 72]. The MOSFHET library [42] adopts this technique. Since scheme switching is much faster than functional keyswitching, it can reduce the number of functional keyswitching from $\ell(k+1)$ to ℓ . The last one is using homomorphic trace evaluation [71, 69] to replace the public functional keyswitching part, improving the Conv. step asymptotically.

B Correctness Proof

To make the noise growth during LUT computation with the Tetris algorithm explicit, we decompose it into two building blocks: circuit bootstrapping (lines 2-7), which performs noise refreshing and ciphertext conversion, and LUT evaluation (lines 8-10).

Theorem 1 (Tetris). *Let $\mathbf{c}_i$ be GLWE ciphertexts of a phase $\Delta m_i + E_{\text{in}}$ under $\mathbf{S}$. During our Tetris algorithm, the circuit bootstrapping building block returns*

⁶To be precise, it requires k private functional keyswitchings and a single public functional keyswitching since $S_{k+1} = -1$.

δ GGSW ciphertext encrypted $m_{i,j}$ with the error $E_{i,j}$, where the variance $E_{i,j}$ is given as follows.

$$\text{Var}(E_{i,j}) \leq c_j \cdot V_{\text{pbs}} + \frac{N}{2} V_{\text{rtr}} + V_{\text{dss}},$$

where $c_0 = c_1 = 1$, $c_j = 2^{j-1} + 1$ when $j > 1$. $V_{\text{pbs}}, V_{\text{rtr}}, V_{\text{dss}}$ is the error variance introduced by *PBSmanyLUT*, *RevHomTrace* and *DecSchemeSwitch*, respectively.

Then, *LUT* evaluation building block returns GLWE ciphertexts of phase $\Delta f(m)_i + E_{\text{out}}$ under the GLWE secret $\mathbf{S}'$ where the variance of E_{out} is given as follows.

$$\text{Var}(E_{\text{out}}) \leq \left(\frac{2N}{q}\right)^2 \cdot \left(i \sum_{j=0}^{\tau-1} (V_{\text{ext},j}) + V_{\text{srks}}\right) + V_{\text{ms}},$$

where $(V_{\text{ext},j})$ is the error variance introduced by the external product with GGSW ciphertext encrypted the j -th bit, V_{srks} and V_{ms} is the error variance introduced by *SR-KeySwitch* and *ModSwitch* respectively.

Proof. We begin by analyzing the noise during circuit bootstrapping.

TetrisRefresh. Given a GLWE ciphertext $\mathbf{c}$ of the phase $\Delta m + E_{\text{in}}$, and bootstrapping key BSK, *PBSmanyLUT* outputs a GLWE ciphertext, where the phase of k -th GLWE ciphertext is

$$\frac{v_k}{2} (-1)^{m_0+1} + y_1 X + \dots + y_{N-1} X^{N-1} + E_{\text{pbs}}(X),$$

The noise variance V_{pbs} of $E_{\text{pbs}}(X)$ is estimated as follows.

$$\begin{aligned} V_{\text{pbs}} = & n \ell_{\text{pbs}} (k+1) N \frac{B_{\text{pbs}}^2 + 2}{12} \sigma_{\text{bsk}}^2 + n \frac{q^2 - B_{\text{pbs}}^{2\ell_{\text{pbs}}}}{24 B_{\text{pbs}}^{2\ell_{\text{pbs}}}} \left(1 + \frac{kN}{2}\right) \\ & + \frac{nkN}{32} + \frac{n}{16} \left(1 - \frac{kN}{2}\right)^2 \end{aligned}$$

where σ_{bsk}^2 is the noise variance of the bootstrapping key BSK.

Then we use MVPBS to get left $\tau - 1$ GLWE ciphertexts by multiply the polynomial $G_j(X)$, so the phase of $\mathbf{C}_j$ is

$$\frac{v_k}{2} (-1)^{m_j+1} + y_1 X + \dots + y_{N-1} X^{N-1} + G_j(X) E_{\text{pbs}}(X),$$

where the variance of error $G_j(X) E_{\text{pbs}}(X)$ is $c_j V_{\text{pbs}}$ and $c_j = \ell_2(G_j)^2 = 2^{j-1} + 1$. **RevHomTrace.** Let E_d be the increased error polynomial after the d -th iteration. Then, E_d satisfies the following relation

$$E_d = \left(\frac{1}{2} E_{d-1} + E_{\text{MS}}\right) + \text{Auto}_{2^d+1} \left(\frac{1}{2} E_{d-1} + E_{\text{MS}}\right) + E_{\text{Auto}},$$

where E_{MS} and E_{Auto} are the error increments by modulus switching and EvalAuto, respectively. So the noise variance satisfies

$$\text{Var}(E_d) \leq \text{Var}(E_{d-1}) + 4E_{\text{MS}} + V_{\text{Auto}}.$$

Finally, after $d = \log N$ iterations and $E_0 = 0$, and $E_0 = \text{Var}(\mathbf{C})$, we get

$$\text{Var}(\mathbf{C}') \leq \text{Var}(\mathbf{C}) + V_{\text{rtr}},$$

where $V_{\text{rtr}} = 4 \log N \cdot V_{\text{MS}} + \log N V_{\text{Auto}}$. **DecSchemeSwitch**. Let $\overline{\mathbf{C}}'$ be a GLev ciphertext of M having variance σ_{in}^2 . Let σ_{ssk}^2 be the noise variance of the scheme switching key. Then, Algorithm **DecSchemeSwitch** returns a GGSW ciphertext $\overline{\mathbf{C}}'$ of M having variance V_{out} such that $V_{\text{out}} \leq \frac{N}{2} \sigma_{\text{in}}^2 + V_{\text{dss}}$ where

$$\begin{aligned} V_{\text{ss}} \leq & \frac{(1 + k'N)N}{2} \left(\frac{q^2 - B_{\text{ss}}^{2\ell_{\text{ss}}}}{24B_{\text{ss}}^{2\ell_{\text{ss}}}} + \frac{1}{12} \right) \\ & + (k' + 1)\ell_{\text{ss}}N \left(\frac{B_{\text{ss}}^2 + 2}{12} \right) \sigma_{\text{ssk}}^2 \end{aligned}$$

In summary, the error variance of the GGSW ciphertext obtained via circuit bootstrapping is

$$\text{Var}(E_{i,j}) \leq c_j \cdot V_{\text{pbs}} + \frac{N}{2} V_{\text{rtr}} + V_{\text{dss}}.$$

Next, we analyze the error propagation during LUT evaluation building block.

LUT Let $\overline{\mathbf{C}}_1 = \text{GGSW}_{\mathbf{S}}^{(B,\ell)}(M)$ be a GGSW ciphertext of M having variance σ_{ext}^2 under $\mathbf{S}'$ and $\mathbf{C}_2$ be a GLWE ciphertext of a phase μ under $\mathbf{S}'$. Then, external product $\overline{\mathbf{C}}_1 \boxtimes \mathbf{C}_2$ outputs a GLWE ciphertext of a phase $\mu \cdot M + E_{\text{ext}}$ under $\mathbf{S}'$ where the variance V_{ext} of E_{ext} is given as follows.

$$\begin{aligned} V_{\text{ext}} \leq & (1 + k'N) \left(\frac{q^2 - B^{2\ell}}{24B^{2\ell}} + \frac{1}{12} \right) \ell_2(M)^2 \\ & + (k' + 1)\ell N \left(\frac{B^2 + 2}{12} \right) \sigma_{\text{ext}}^2 \end{aligned}$$

Since LUT evaluation is composed with the external product between GLWE and GGSW ciphertext, and $\ell_2(M)^2 \leq 1$. The noise increment of LUT evaluation using the i -th digit is

$$\text{Var}(E_{\text{out}}) \leq i \sum_{j=0}^{\tau-1} (V_{\text{ext},j}),$$

where the $V_{\text{ext},j}$ represents the error variance increase induced by the j -th GGSW external product.

SR – KeySwitch Let $\mathbf{C}$ be a LWE ciphertext of a phase μ under $\mathbf{S}_1$. Let $\sigma_{\mathbf{S}_1 \rightarrow \mathbf{S}_2}^2$ be the noise variance of the LWE keyswitching key from $\mathbf{S}_1$ to $\mathbf{S}_2$, where $\mathbf{S}_1 =$

$\mathbf{S}_2 || \mathbf{S}_3$. Then, Algorithm 5 returns a LWE ciphertext $\mathbf{C}'$ of a phase $\mu + E_{\text{ks}}(X)$ under $\mathbf{S}_2$ where the variance V_{ks} of $E_{\text{ks}}(X)$ is given as follows.

$$\begin{aligned} \text{Var}(E_{\text{ks}}) &\leq (kN - n) \left(\frac{q^2 - B^{2\ell}}{24B^{2\ell}} + \frac{1}{12} \right) \\ &\quad + \ell(kN - n) \left(\frac{B^2 + 2}{12} \right) \sigma_{\mathbf{S}_1 \rightarrow \mathbf{S}_2}^2. \end{aligned}$$

ModSwitch Let $\mathbf{C} \in \mathcal{R}_{q,N}^{k+1}$ be a GLWE ciphertext of a phase μ under $\mathbf{S}$. Then, modulus switching outputs a GLWE ciphertext $\mathbf{C}' \in \mathcal{R}_{q',N}^{k+1}$ of a phase $\frac{q'}{q}\mu + E_{\text{ms}}$ under $\mathbf{S}$ where the variance V_{ms} of E_{ms} is given as follows.

$$V_{\text{ms}} \leq \frac{kN + 4}{48}.$$

In summary, the error variance of the LUT evaluation is

$$\text{Var}(E_{\text{out}}) \leq \left(\frac{2N}{q} \right)^2 \cdot \left(i \sum_{j=0}^{\tau-1} (V_{\text{ext},j}) + V_{\text{srks}} \right) + V_{\text{ms}},$$

□

In our Tetris algorithm, TetrisRefresh resets the noise of the input ciphertext to a low level. After it, all subsequent operations contribute to noise growth until the next TetrisRefresh is applied. As a result, the ciphertext with the maximum noise throughout the entire computation occurs after the **ModSwitch** and prior to entering the next TetrisRefresh.

At this point, suppose that GLWE ciphertext after **ModSwitch** of noise variance σ_{ms}^2 . We estimated the probability of decryption failure as $P = 1 - \text{erf} \left(\frac{2N \cdot 2^{-\vartheta}}{2^{\tau+1} \sqrt{2} \sigma_{\text{ms}}} \right)$.

C Why Tetris Benefit from Noise Management

TFHE PBS (including PBSmanyLUT in CMux-tree method) involves evaluating the LWE decryption circuit homomorphically to refresh the noise in ciphertexts. Recall that the LWE decryption involves

$$b - \sum_{i=0}^{n-1} a_i s_i \mod q,$$

in which a modular reduction must be executed homomorphically. To this end, TFHE encodes elements $a \in \mathbb{Z}_q$ as monomials $X^a \mod (X^N + 1)$, thereby embedding the computation of $(\mathbb{Z}_q, +)$ into the multiplicative cyclic group $\{1, X, \dots, X^{2N-1}\}$. To ensure this algebraic mapping, TFHE must guarantee that switching the ciphertext modulus to $2N$ does not distort the correctness of the embedded plaintext. In other words, the encoding capacity of PBS is bounded by $2N$.

Plaintext Encoding in TFHE. Within this $2N$ -sized encoding space, the TFHE scheme utilizes the most significant bit encoding (MSB) strategy following several partitioning paradigm. The encoding space can be divided into five consecutive regions from most significant to least significant bits to cover different TFHE evaluation methods:

- **Padding Space** (S_p , if needed): When performing TFHE LUTs using PBS or PBS-tree, it typically requires one padding bit of zero to evaluate arbitrary functions as explained in Section 3. In contrast, Evaluations based on CMux-tree do not incur this limitation.
- **Message Space** (S_m): Contains plaintext messages scaled by a factor Δ and encoded in the most significant bits after the padding bit.
- **Buffer Space** (S_b): Acts as guard bits between the message and noise. The size is determined by the required decryption failure probability, calculated through tail analysis of the Gaussian noise distribution.
- **Noise Space** (S_e): Represents the standard deviation of the noise. Because the noise follows a Gaussian distribution, it can spill into more significant positions, which explains the necessity of a buffer space.

Evidently, under the $2N$ -bounded encoding described above, noise shrinking enables either (i) encoding more message bits without changing N , or (ii) reducing N while keeping the message space unchanged. Since fully homomorphic evaluation primarily performs additions and multiplications over polynomial rings, the polynomial dimension N has a pronounced impact on computational efficiency. This explains why Tetris attains substantial performance improvements from noise shrinking.

D Supplemental Materials: Tables and Figures

Table 7: Instruction Set Description

Instruction	Function
ADD	Add the two inputs together.
SUB	Subtract the second input from the first.
MULL	Multiply two inputs and return the lower n bits of the result.
MULH	Multiply two inputs and return the upper n bits of the result.
DIV	Divide the first input by the second as an unsigned integer.
MOD	Compute the remainder of the first input divided by the second.
EQ	Return 1 if the first input equals the second; otherwise return 0.
GT	Return 1 if the first input is greater than the second; otherwise return 0.
LT	Return 1 if the first input is less than the second; otherwise return 0.
GTE	Return 1 if the first input is greater than or equal to the second; otherwise return 0.
LTE	Return 1 if the first input is less than or equal to the second; otherwise return 0.
MAX	Return the maximum of the two inputs.
MIN	Return the minimum of the two inputs.
RL	Rotate the first input left by the number of bits given by the second input.
RR	Rotate the first input right by the number of bits given by the second input.
SL	Shift the first input left by the number of bits given by the second input.
SR	Shift the first input right by the number of bits given by the second input.
OR	Bitwise OR of the two inputs.
AND	Bitwise AND of the two inputs.
XOR	Bitwise XOR of the two inputs.
NAND	Bitwise NAND of the two inputs.
MOVE	Move the data from the input position to the output position.
SIGN	Return 1 if the first input is greater, 0 if equal, and -1 if less.

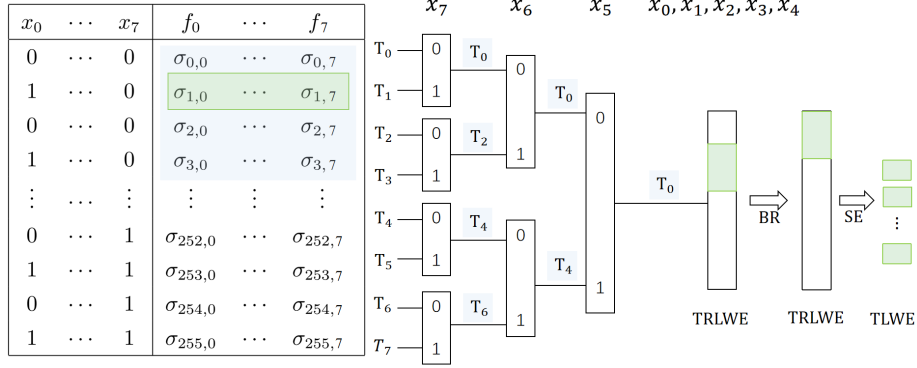


Figure 8: 8-to-8-bit, $\tau = 1$ lookup table using mixed packing: first enumerate the function’s truth table; then every $32 \times 8 = 256$ function values (blue region) are packed into a single GLWE ciphertext, producing eight ciphertexts encrypting $T_0, \dots, T_7$ (for illustration; in the actual algorithm all GLWE slots are fully utilized). To look up the output for the binary input 10000000, the first three bits select ciphertext T_0 , and the remaining five bits’ GGSWs blind-rotate the corresponding function value into the first eight coefficients of the GLWE. Finally, sample extraction returns the eight LWE ciphertexts.

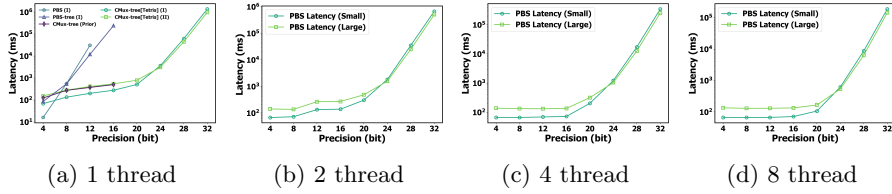


Figure 9: Multiple Threads Performance Evaluation.

Table 8: Tetris Performance for Differet Precission

Precision	Threads	Set I			Set II		
		CBS Time	LUT Time	Total Time	CBS Time	LUT Time	Total Time
4	1	68.117	0.416	68.533	152.363	0.253	152.616
4	2	66.409	0.439	66.848	140.347	0.252	140.599
4	4	66.199	0.430	66.630	137.685	0.260	137.945
4	8	66.659	0.462	67.121	135.484	0.262	135.746
8	1	133.695	1.100	134.795	283.121	0.716	283.837
8	2	70.699	1.066	71.765	135.426	0.625	136.051
8	4	64.899	1.270	66.168	132.920	0.757	133.677
8	8	64.647	2.213	66.859	130.102	0.726	130.828
12	1	198.685	2.747	201.432	411.098	1.716	412.814
12	2	130.822	2.483	133.305	262.640	1.561	264.201
12	4	66.828	2.171	68.999	130.500	1.315	131.815
12	8	64.978	2.250	67.228	129.975	1.441	131.416
16	1	265.324	12.276	277.601	537.859	7.940	545.799
16	2	130.951	7.113	138.064	264.972	4.475	269.447
16	4	67.343	4.762	72.105	132.992	2.854	135.846
16	8	65.937	5.680	71.617	131.065	3.694	134.759
20	1	336.974	173.658	510.632	668.595	121.625	790.220
20	2	197.243	104.491	301.734	406.516	74.436	480.952
20	4	131.757	70.515	202.271	266.708	49.948	316.656
20	8	66.250	40.762	107.012	138.434	28.414	166.848
24	1	417.784	3220.379	3638.163	809.702	2265.363	3075.065
24	2	203.360	1608.193	1811.552	428.941	1134.816	1563.757
24	4	145.977	1074.420	1220.397	291.833	754.761	1046.594
24	8	74.214	548.529	622.743	148.875	389.374	538.249
28	1	541.802	60999.767	61541.569	1067.860	43087.494	44155.354
28	2	324.064	34623.989	34948.053	675.111	24125.092	24800.203
28	4	183.580	17031.021	17214.601	383.613	12157.134	12540.747
28	8	128.874	8677.189	8806.063	245.765	6158.355	6404.120
32	1	623.566	1359845.429	1360468.995	1526.129	963256.309	964782.438
32	2	386.842	666078.599	666465.441	812.124	504077.144	504889.268
32	4	202.277	345669.640	345871.917	526.950	253207.612	253734.562
32	8	163.445	187524.820	187688.265	383.291	143074.021	143457.312

(a) Comprehensive evaluation of Tetris: latency across steps, precision, parameter sets, and parallelism.

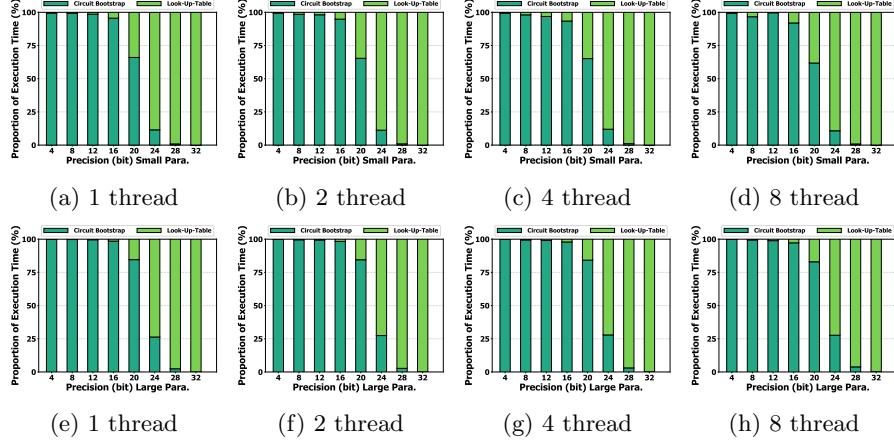


Figure 10: Multiple Threads CBS vs LUT Performance Evaluation

Table 9: Instructions Performance Evaluation

	TetrISA		PC	TCHES'25	
	CC	CP		CC	CP
ADD	291.44ms	144.09ms	131.93ms	1.15s	0.30s
SUB	266.98ms	130.61ms	129.73ms	1.15s	0.31s
MULL	262.89ms	129.62ms	131.47ms	1.66s	0.30s
MULH	260.97ms	129.11ms	129.81ms	5.58s	0.48s
DIV	259.11ms	129.02ms	128.87ms	16.82s	0.50s
MOD	260.80ms	125.47ms	126.96ms	15.59s	0.49s
EQ	259.78ms	127.32ms	127.90ms	0.93s	0.25s
GT	260.50ms	126.02ms	126.74ms	1.45s	0.28s
LT	257.22ms	127.12ms	127.68ms	1.44s	0.28s
GTE	263.48ms	131.07ms	129.71ms	1.46s	0.28s
LTE	259.08ms	128.15ms	127.76ms	1.45s	0.28s
MAX	259.98ms	128.49ms	128.79ms	2.75s	0.49s
MIN	263.01ms	127.29ms	128.16ms	2.72s	0.44s
RL	260.79ms	128.01ms	126.46ms	1.58s	0.30s
RR	260.84ms	129.06ms	128.57ms	1.59s	0.30s
SL	261.84ms	129.28ms	129.68ms	1.05s	0.23s
SR	260.24ms	130.79ms	128.57ms	1.04s	0.25s
OR	261.23ms	129.56ms	130.08ms	0.62s	0.15s
AND	262.12ms	128.84ms	129.56ms	0.59s	0.15s
XOR	259.43ms	128.60ms	129.21ms	0.61s	0.15s
NAND	261.46ms	128.48ms	129.52ms	-	-
MOVE	-	1.77 μ s	-	-	-

(a) Performance comparison of instructions between TetrISA and Trama et al. [67] under 8-bit bivariate / univariate setting.